**CPCS241-Database I**

# Home Health Care Centre Database

## Problem Definition and Analysis

**Group No:** *1*

**Group leader: Rahaf**

| Student Name | Student Number |
|---|---|
| **Rahaf** | |
| **Layal** | |
| **Nawal** | |
| **Linah Basim Alsubhi** | |
| **Mashael** | |

# Contents

PART I: Analysis ..... 3
1 Problem Definition and Data Requirements ..... 3
1.1 Problem Description ..... 3
1.2 Data Requirements ..... 4
1.3 Business Rules ..... 6
PART II: DB DESIGN ..... 7
2 ER Diagram Design ..... 7
2.1 ER diagram ..... 7
2.2 Design of Business Rules ..... 8
3 ER-to-logical schema mapping ..... 10
3.1 Mapping of Regular Entity Types ..... 10
3.2 Mapping of Weak Entity Types ..... 15
3.3 Mapping of binary 1-1 relationship Types ..... 16
3.4 Mapping of binary 1-N relationship Types ..... 17
3.5 Mapping of binary M-N relationship types ..... 21
3.6 Mapping of multivalued attributes ..... 23
3.7 Mapping of N-Ary relationship types ..... 25
3.8 Schema Diagram ..... 25
4 Normalization ..... 30
4.1 First Normal Form ..... 30
4.2 Second Normal Form ..... 32
4.3 Third Normal Form ..... 35
5 Final DB Schema Diagram ..... 39
6 Table Creation Script ..... 43
6.1 <*PATIENT_EMERGENCY*> TABLE ..... 43
6.2 <*PATIENT*> TABLE ..... 43
6.3 <*PATIENT_ALLERGIES*> TABLE ..... 44
6.4 <*PATIENT_CHRONIC_ILLNES*> TABLE ..... 44
6.5 <*TREATMENT_PLAN_MEDICATION* > TABLE ..... 45
6.6 <*MEDICATION*> TABLE ..... 45
6.7 <*TREATMENT_PLAN*> TABLE ..... 46
6.8 <*DIAGNOSIS*> TABLE ..... 46
6.9 <*EMPLOYEE*> TABLE ..... 47
6.10 <*DOCTOR*> TABLE ..... 47
6.11 <*DEPARTMENT*> TABLE ..... 48

6.12 <*NURSE*> TABLE ..... 48
6.13 <*VEHICLE_DRIVER*> TABLE ..... 49
6.14 <*VEHICLE*> TABLE ..... 50
6.15 <*VEHICLE_AND_DRIVER*> TABLE ..... 50
6.16 <*SERVICE*> TABLE ..... 51
6.17 <*BILL*> TABLE ..... 51
6.18 <*APPOINTMENT*> TABLE ..... 52
6.19 <*APOINTMENT_BILL*> TABLE ..... 53
6.20 <*MEDICATION_APPOINTMENT*> TABLE ..... 53
6.21 <*APPOINTMENT_SERVICE*> TABLE ..... 54
6.22 <*DOCTOR_NURSE*> TABLE ..... 54
6.23 <*EMPLOYEE_AVAILABLE_SCHEDULE*> TABLE ..... 55
6.24 <*NURSE_QUALIFICATION*> TABLE ..... 56
6.25 <*SERVICE_EQUIPMENT*> TABLE ..... 56
6.26 <*DOCTOR_ACADEMIC_DEGREE*> TABLE ..... 57
7 Constraints Script ..... 57
8 Queries ..... 62
APPENDIX ..... 69

# PART I: Analysis

## 1 Problem Definition and Data Requirements

### 1.1 Problem Description

In this project, a database is designed for the Home Health Care Centre. Home Health Care Centre is recognized for improving the health of individuals and families. It

specializes in providing medical care directly to patients' homes. It manages a team of well-trained doctors and nurses, aiming to serve the needs of the patients.

It is required to design a database to keep track of the Centre's Patients, doctors, nurses, treatment plans, and appointments.

## 1.2 Data Requirements

**Patient:**

1. Each patient has a unique ID and a distinct medical file number.

2. The patient's name (first and last), date of birth must be stored. And gender, contact information (phone number, Email) and address (postal code, district, street) are stored.

3. Each patient must have at least one eligibility criteria (e.g., disability, geriatric, overweight, mental illness).

4. Each patient have one or more emergency contacts (name, relationship, and phone number).

5. the patient's chronic illnesses and allergies are recorded.

**Doctor:**

1. Each doctor has a unique ID number and employee number.

2. The doctor's name, date of birth, gender, academic degree, specialization, contact information (phone number, Email), and available schedule (days, time, duration, year) are recorded.

**Nurse:**

1. Each nurse has a unique ID number and employee number.

2. The nurse's name, date of birth, gender, qualifications, contact information (phone number, Email), and available schedule (days, time, duration, year) are recorded.

**Department:**

1. Each department has a unique name and its supervising doctor(s) and supervising nurse(s).

**Diagnosis:**

1. Each diagnosis has a unique code name.

2. The diagnosis stores the chief complaint, diagnosis name and clinical notes including the patient's file number.

**Treatment plan:**

1. Each treatment plan has a unique treatment ID.

2. Each treatment plan includes the treatment starting date and expected duration.

3. The treatment plan stores the treatment description and medications.

**Service:**

1. Each service has a unique Service ID.

2. Each service has a name, description, medical equipment, estimated duration, and price.

**Appointment:**

1. Each appointment has a unique Appointment ID.

2. Appointments are linked to the patient's file number, assigned doctors and nurses, the relevant service (Service ID), and transportation if applicable (Vehicle ID).

3. The appointment includes the date, time, and status (e.g., Scheduled, Completed, Cancelled).

**Medication:**

1. Each medication has a unique Medication ID.

2. Each prescription is linked to a Prescription ID, patient's file number, and prescribing doctor.

3. The medication includes its name, usage instructions, and refill availability (Yes/No).

**Vehicle:**

1. Each vehicle has a unique Vehicle ID.

2. The vehicle's plate number, assigned driver (Driver ID), and operational status (e.g., Available, In Use, Needs Maintenance) are tracked.

**Vehicle Driver:**

1. Each driver has a unique employee number and ID number.

2. Each Vehicle Driver has a name, date of birth, gender, and contact information (phone number, Email) and their available schedule (days, time, duration, year).

**Bills:**

1. Each bill has a unique Bill ID.

2. Billing details are tracked and linked to the patient's file number and the services provided (Service ID).

3. The bill includes the total cost, payment status (e.g., Paid, Unpaid, Pending), and selected payment method.

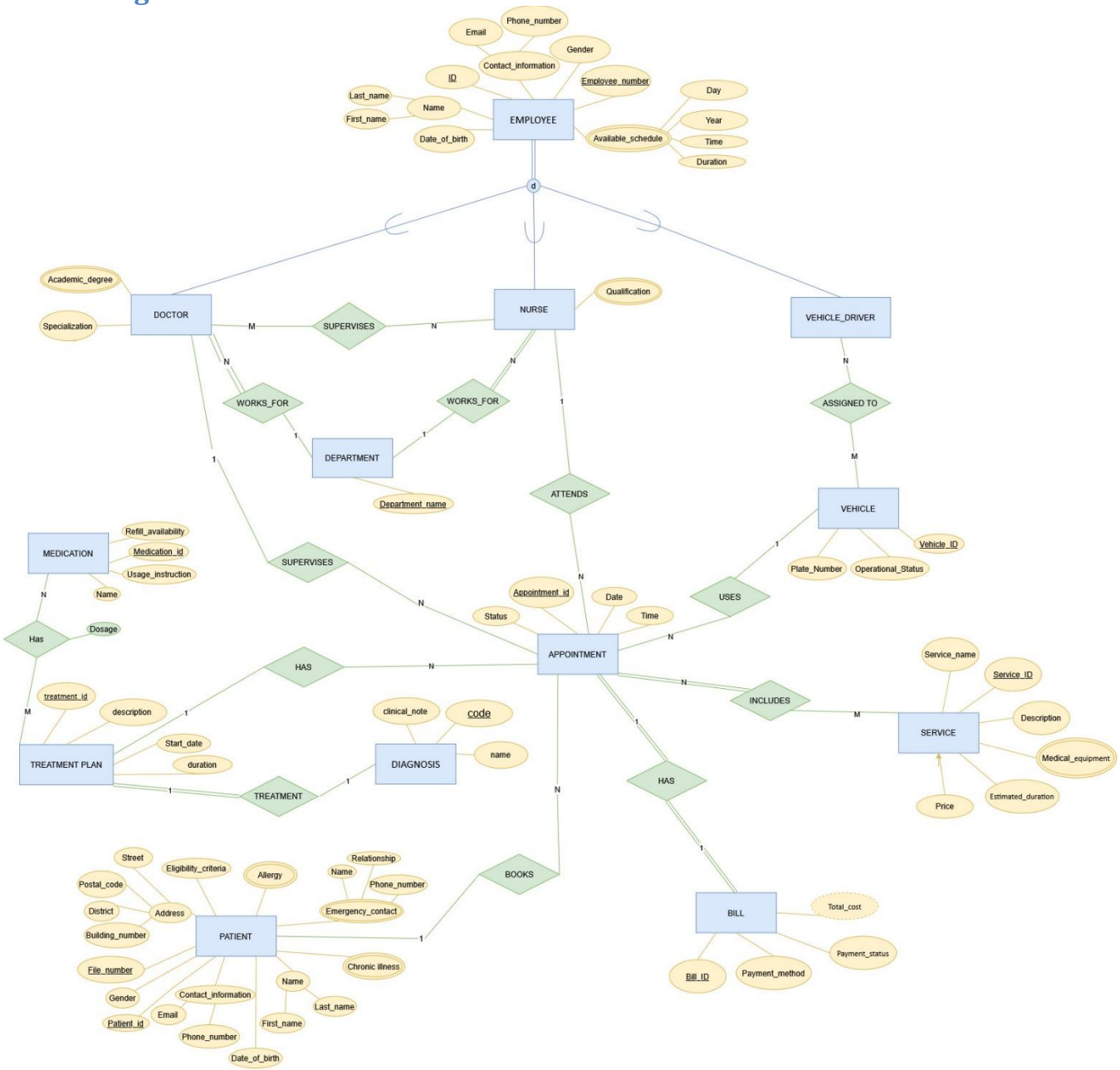
## 1.3 Business Rules

1. Each patient can book multiple appointments.
2. Each appointment is booked for one patient.
3. A doctor supervises multiple appointments.
4. An appointment can have only one doctor.
5. A nurse attends multiple appointments
6. An appointment can have only one nurse.
7. Each appointment must have one or more services.
8. Each service is included in many appointments.
9. Each appointment must have one bill.
10. Each bill must be associated with only one appointment.
11. Each vehicle can be assigned to multiple drivers.
12. Each driver can be assigned to multiple vehicles.
13. Each appointment can use only one vehicle.
14. Each vehicle can be used for multiple appointments.
15. Each treatment plan can prescribe one or more medications, and for each prescription dosage is recorded.
16. Each medication can be prescribed in one or more treatment plans, and for each prescription dosage is recorded.
17. Each doctor must work for one department.
18. A department can have multiple doctors working for it.
19. Each nurse must work for one department.
20. A department can have multiple nurses working for it.
21. A doctor supervises multiple nurses
22. A nurse is supervised by multiple doctors.
23. Each treatment plan can be on one or more appointments.
24. Each appointment has only one treatment plan.
25. Each treatment plan must be related to one diagnosis.
26. Each diagnosis linked to one treatment plan.

# PART II: DB DESIGN

## 2 ER Diagram Design

### 2.1 ER diagram



## 2.2 Design of Business Rules

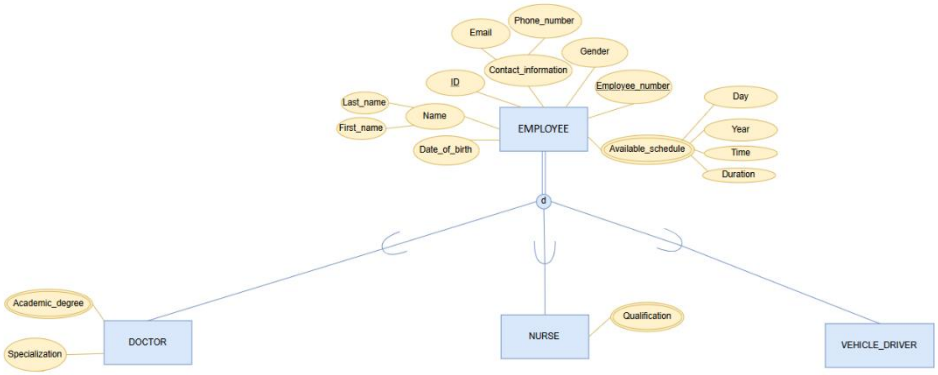
| Entity | Business Rule | Design Decisions | Justification (if any) |
|---|---|---|---|
| **Patient** | Each patient can book multiple appointments | 1-N Binary Relationship between the total patient and appointment | The patient may have one or more appointments |
| **Doctor** | A doctor supervises multiple appointments | 1-N Binary relationship between total doctors and appointments | The doctor may supervise one or more appointments |
| | Each doctor must work for one department | 1-N Binary Relationship between total doctors and the department | The doctor must work for one department only |
| | A doctor supervises multiple nurses | N-M Binary Relationship between total doctors and nurses | The doctor supervises more than one nurse |
| **Nurse** | A nurse is supervised by multiple doctors | N-M Binary Relationship between doctors and nurses | The nurse is supervised by one or more doctors |
| | A nurse attends multiple appointments | 1-N Binary Relationship between nurses and appointments | The nurse can attend more than one appointment |
| | Each nurse must work for one department. | 1-N Binary Relationship between nurses and the department | All the nurses must work for only one department |
| **Department** | A department can have multiple doctors working for it. | 1-N Binary Relationship between the department and multiple doctors. | Each department lists all the doctors working within the same department. |
| | A department can have multiple nurses working for it. | 1-N Binary Relationship between the department and multiple nurses | Each department lists all the nurses working within the same department. |
| **Service** | Each service is included in many appointments. | N-M Binary Relationship between service and appointment | Patients receive medical services during appointments multiple times. |
| **Appointment** | Each appointment is booked for one patient. | 1-N Binary Relationship between appointment and patient | Patients attend multiple appointments over time |
| | An appointment can have only one doctor. | 1-N Binary Relationship between appointment and doctor | Each appointment is associated with exactly one supervising doctor |
| | An appointment can have only one nurse. | 1-N Binary Relationship between an appointment and a nurse | Each appointment is associated with exactly one nurse |

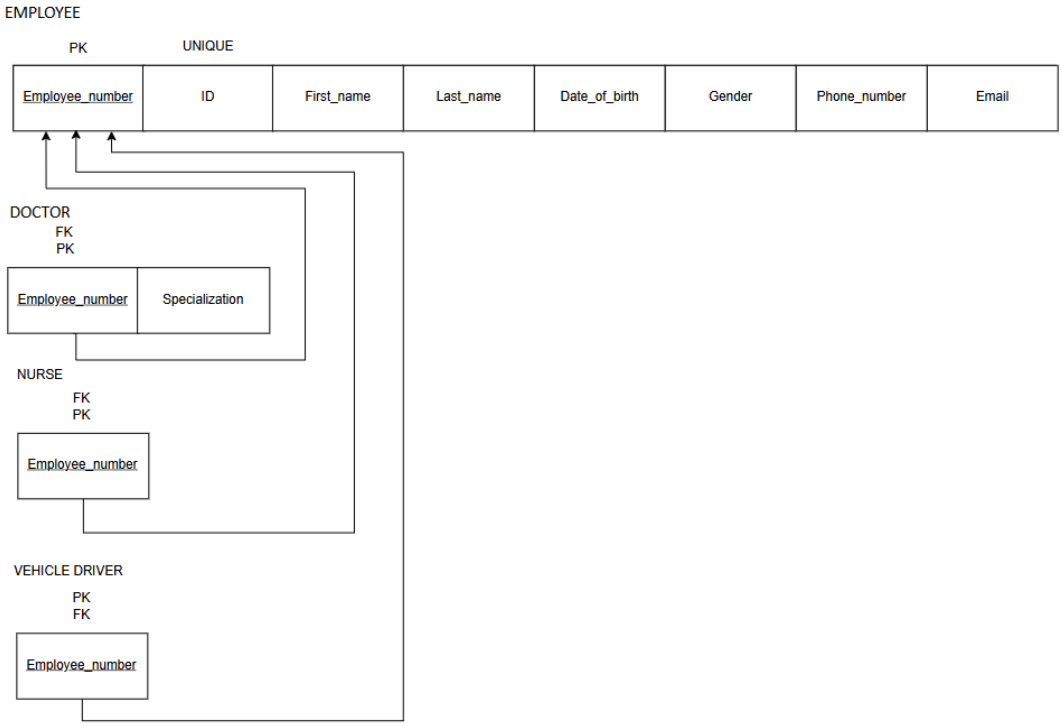
| | | | |
|---|---|---|---|
| | Each appointment must have one or more services. | N-M Binary Relationship between appointment and service | Appointments often involve multiple services provided in one visit. |
| | Each appointment must have one bill. | 1-1 Binary Relationship between appointment and bill | Each appointment generates a single bill. |
| | Each appointment can use only one vehicle | 1-N Binary Relationship between appointment and vehicle | Appointments require transportation to the patient's home |
| | Each appointment has only one treatment plan. | 1-N Binary Relationship between appointment and treatment plan | The appointment is linked to a treatment plan. |
| **Medication** | Each medication can be prescribed in one or more treatment plans, and for each prescription a dosage is recorded. | N-M Binary Relationship with treatment plan. | Medications could be assigned to a treatment plan, but not necessarily. |
| **Vehicle** | Each vehicle can be assigned to multiple drivers. | N-M Binary Relationship with vehicle driver. | Each vehicle could be driven by many drivers at different times. |
| | Each vehicle can be used for multiple appointments. | 1-N Binary Relationship with appointment. | A single vehicle is used to transport all the medical staff and medical equipment. |
| **Vehicle Driver** | Each driver can be assigned to multiple vehicles. | N-M Binary Relationship with vehicle. | Many drivers could drive a specific vehicle at different times. |
| **Bill** | Each bill must be associated with only one appointment. | 1-1 Binary Relationship between total bill and total appointment | To ensure accurate billing and avoid confusion, each bill should match only one appointment. |
| **Diagnosis** | Each diagnosis is linked to one treatment plan. | 1-1 Binary Relationship between total treatment plan and diagnosis | Each diagnosis leads to one specific treatment plan to ensure clarity and personalized care. |
| **Treatment Plan** | Each treatment plan can be on one or more appointments. | 1-N Binary Relationship between treatment plan and appointments | One treatment plan requires several appointments; each appointment follows one plan. |
| | Each treatment plan can prescribe one or more medications, and for each | N-M Binary Relationship with medication.<br>A relationship attribute of the N-M Binary Relationship between | Some patients may need medications in their treatment plan. And each one has a specific dosage determined by a |

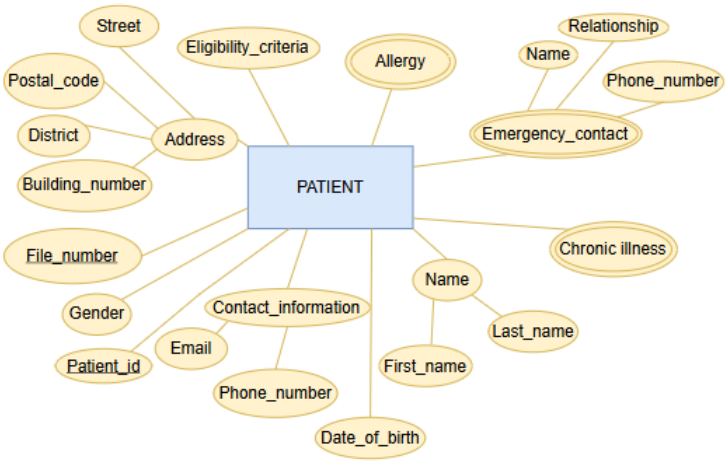
| | | | |
|---|---|---|---|
| | prescription dosage is recorded. | treatment-plan and medication. | supervising doctor for that patient. |
| | Each treatment plan must be related to one diagnosis. | 1-1 Binary Relationship between treatment plan and diagnosis | Each diagnosis has one matching treatment plan for accurate and focused care. |

# 3 ER-to-logical schema mapping

## 3.1 Mapping of Regular Entity Types

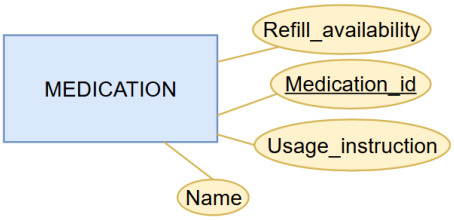





PATIENT

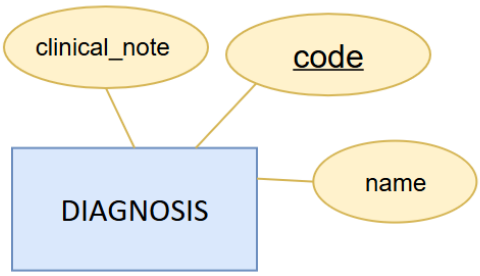
| PK | UNIQUE | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| File_number | Patient_ID | First_name | Last_name | Date_of_birth | Email | Phone_number | gender | District | Street | Building_number | Postal_code | Eligibility_criteria |




MEDICATION

PK

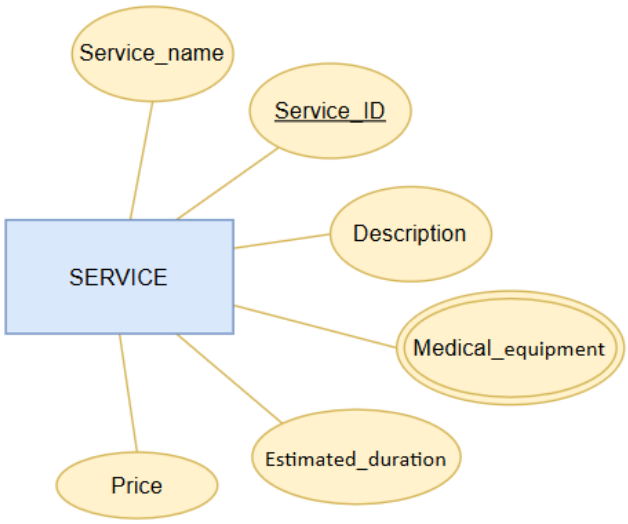
| | | | |
|---|---|---|---|
| Medication_id | Name | Usage_instruction | Refill_availability |




DIAGNOSIS

PK

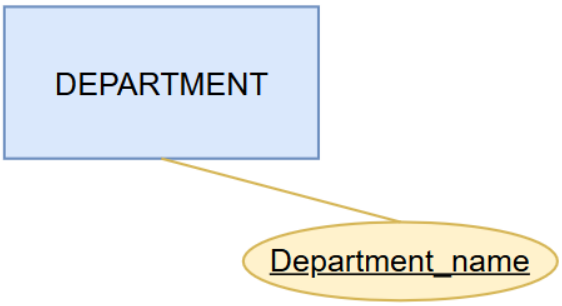
| code | name | clinical_note |
|---|---|---|




SERVICE

PK

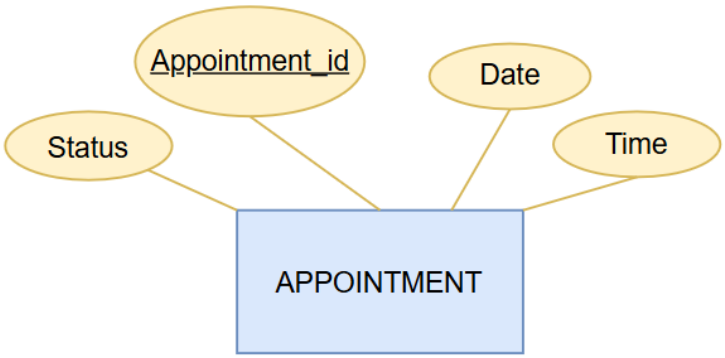
| Service_ID | Service_name | Description | Estimated_duration | Price |
|---|---|---|---|---|




DEPARTMENT

PK

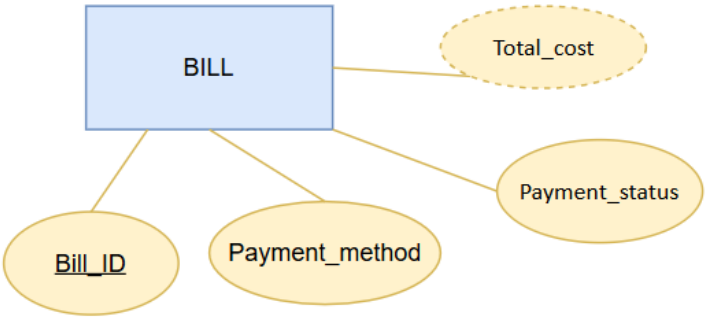
| Department_name |
|---|




APPOINTMENT

PK

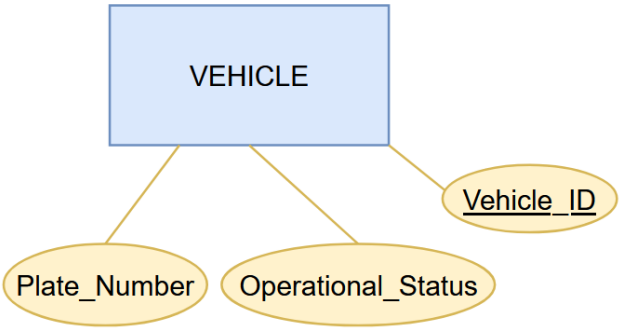
| Appointment_id | Status | Date | Time |
|---|---|---|---|




BILL

PK

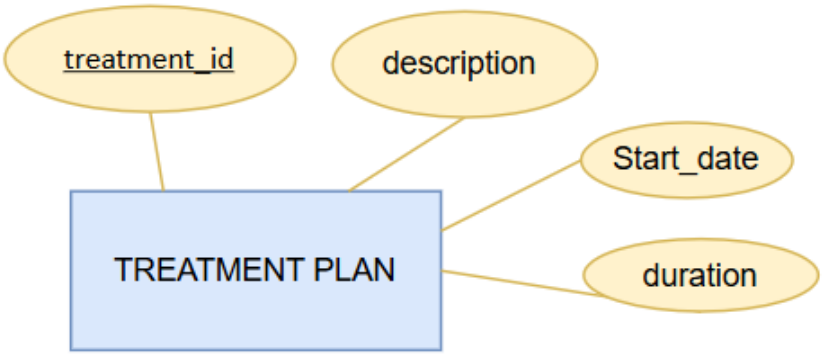
| Bill_ID | Payment_method | Payment_status | total_cost |
|---|---|---|---|




VEHICLE

PK

| Vechile_ID | Operational_Status | Plate_Number |
|---|---|---|




TREATMENT_PLAN

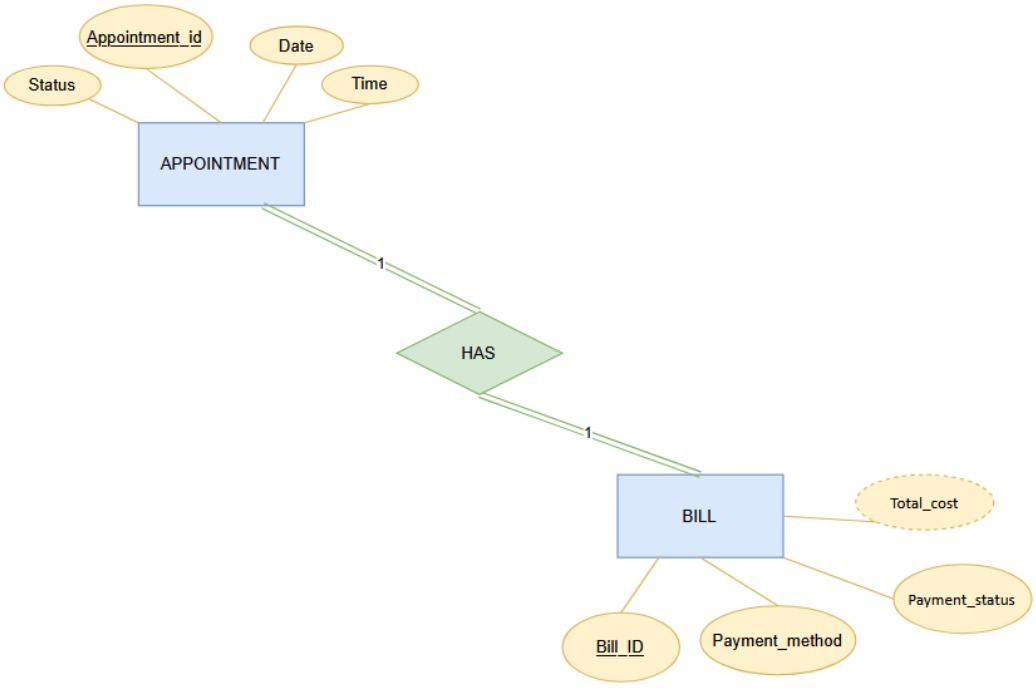
PK

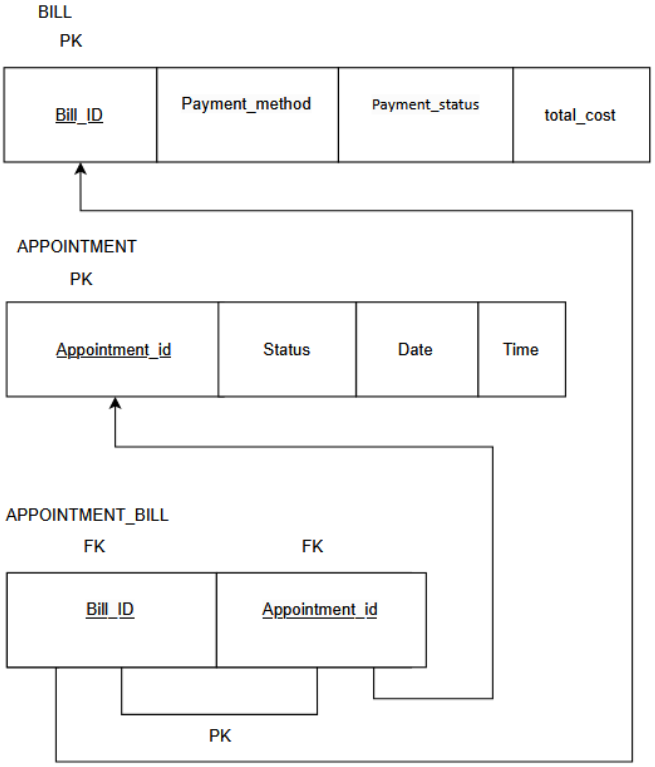
| treatment_id | description | start_date | duration |
|---|---|---|---|

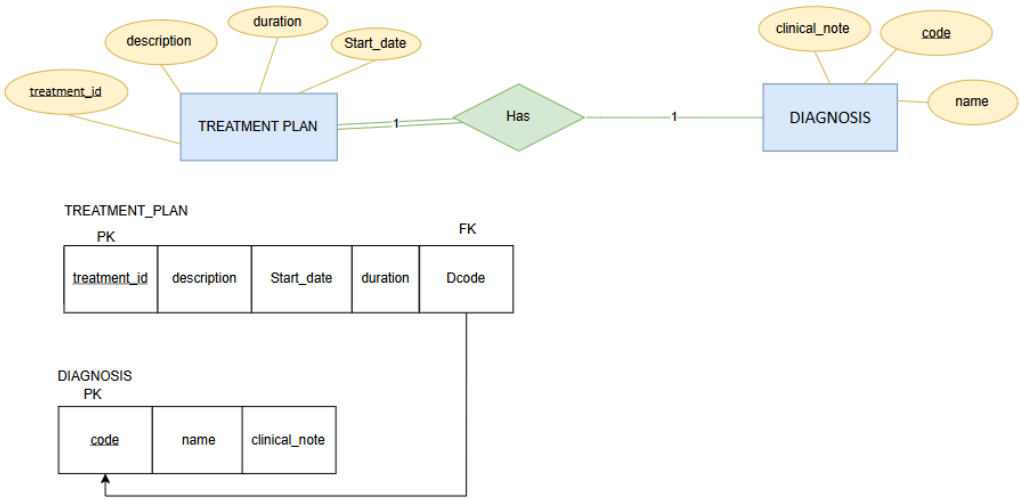
## 3.2 Mapping of Weak Entity Types

The System does not have any weak entities.

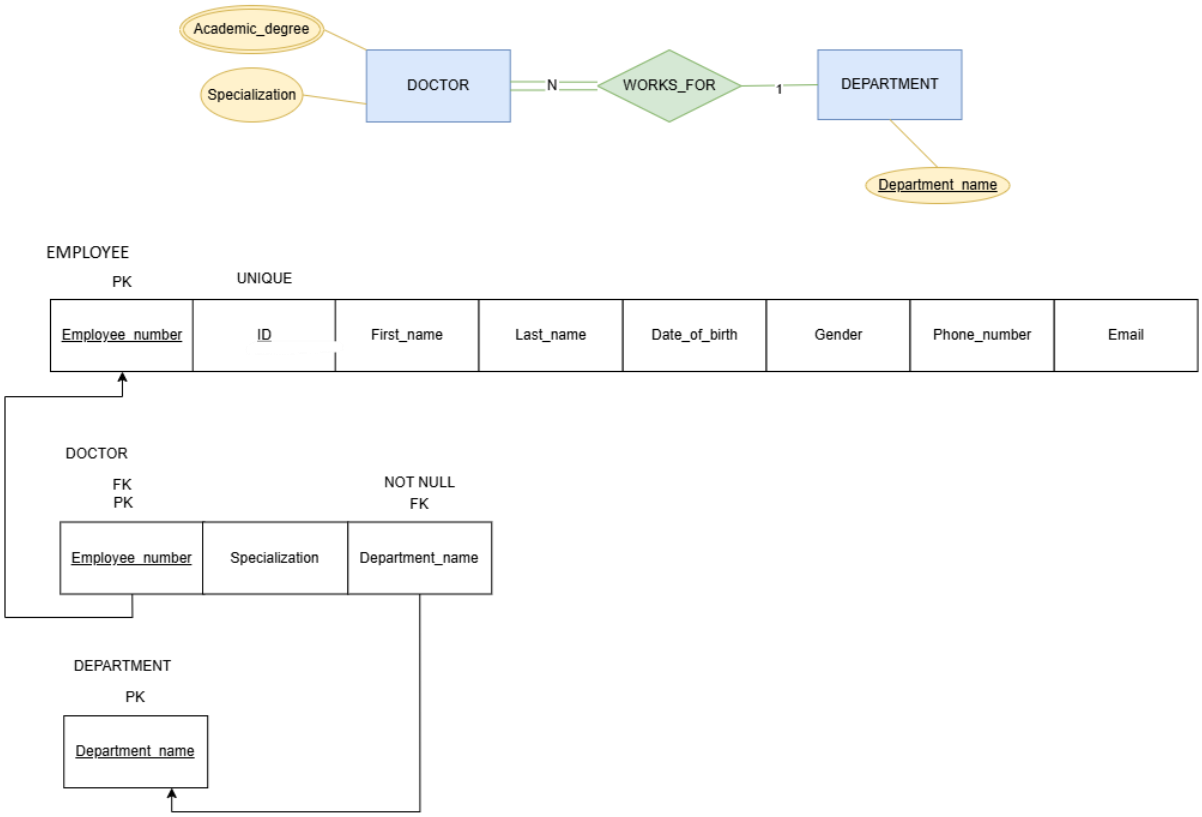
## 3.3 Mapping of binary 1-1 relationship Types

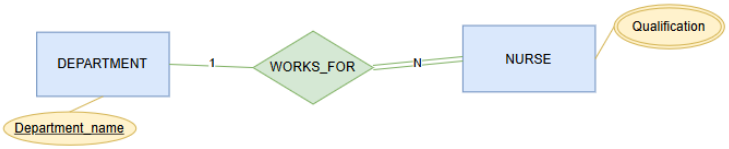


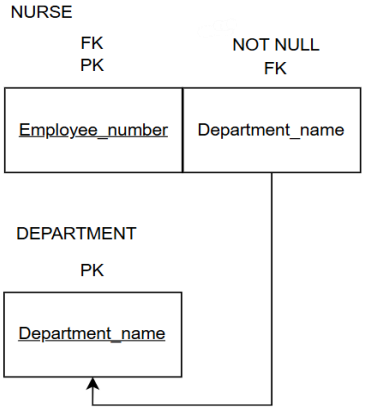



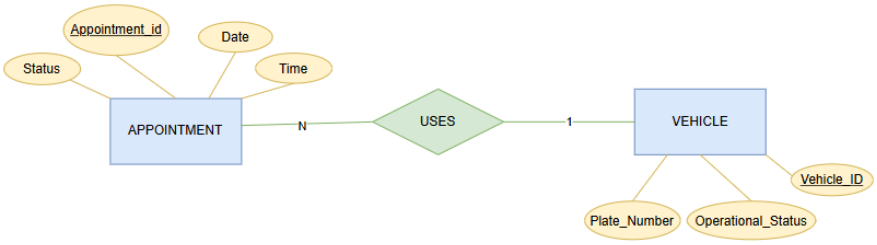


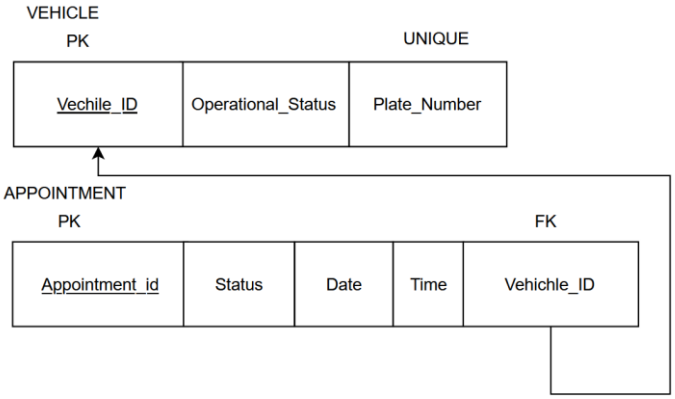

## 3.4 Mapping of binary 1-N relationship Types

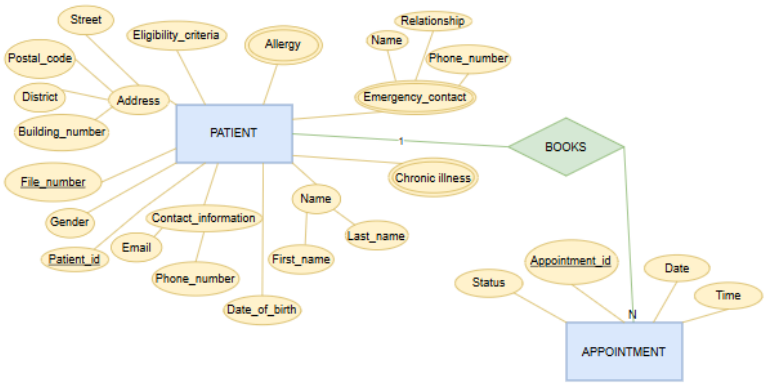



DEPARTMENT
1
WORKS_FOR
N
NURSE
Qualification
Department_name


NURSE
FK
PK
NOT NULL
FK
Employee_number
Department_name
DEPARTMENT
PK
Department_name


Appointment_id
Date
Status
Time
APPOINTMENT
N
USES
1
VEHICLE
Vehicle_ID
Plate_Number
Operational_Status


VEHICLE
PK
UNIQUE
Vechile_ID
Operational_Status
Plate_Number
APPOINTMENT
PK
FK
Appointment_id
Status
Date
Time
Vehichle_ID

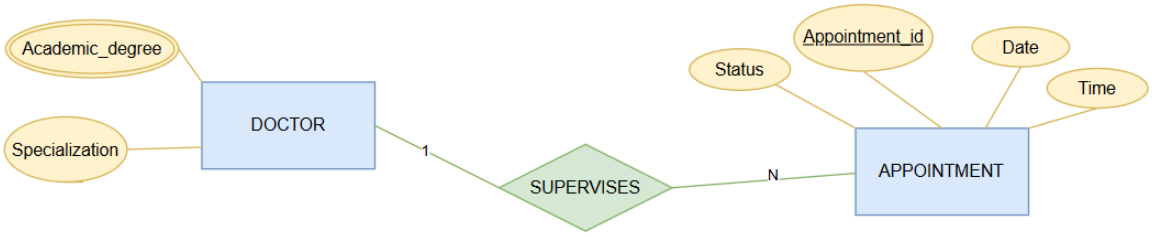



PATIENT

| PK | UNIQUE | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| File_number | Patient_ID | First_name | Last_name | Date_of_birth | Email | Phone_number | gender | District | Street | Building_number | Postal_code | Eligibility_criteria |

APPOINTMENT

| PK | | | | FK | FK |
|---|---|---|---|---|---|
| Appointment_id | Status | Date | Time | Veichle_ID | Patient_number |

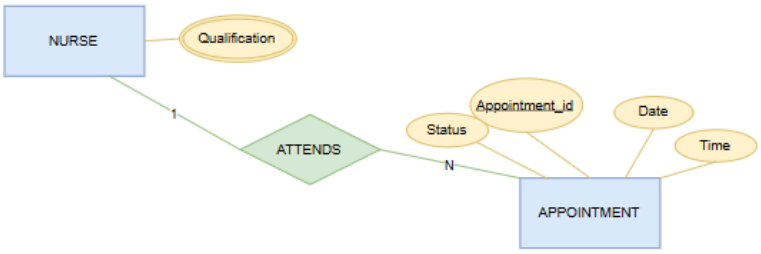



APPOINTMENT

| PK | | | | FK | FK | FK |
|---|---|---|---|---|---|---|
| Appointment_id | Status | Date | Time | Patient_number | DEmployee_number | Veichle_ID |

DOCTOR

| FK<br>PK | | NOT NULL<br>FK |
|---|---|---|
| Employee_number | Specialization | Department_name |

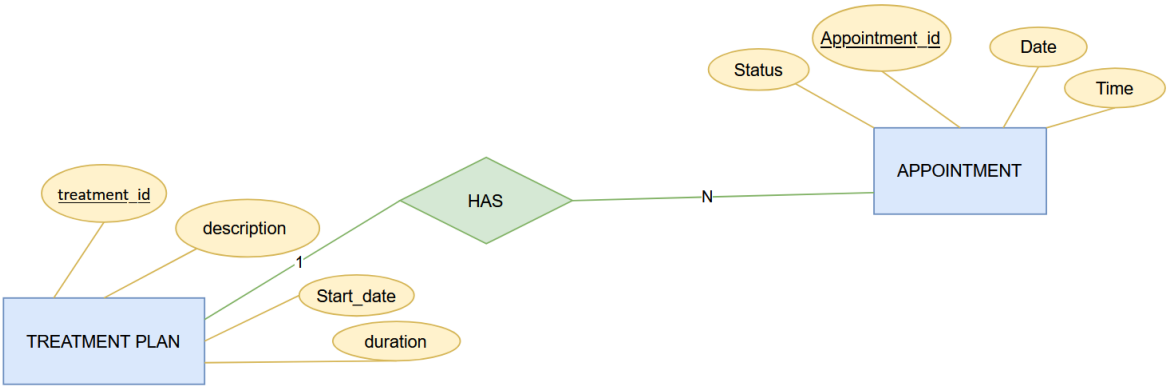



NURSE

| FK<br>PK | NOT NULL<br>FK |
|---|---|
| Employee_number | Department_name |

APPOINTMENT

| PK | | | | FK | FK | FK | FK |
|---|---|---|---|---|---|---|---|
| Appointment_id | Status | Date | Time | Patient_number | DEmployee_number | NEmployee_number | Vehicle_ID |



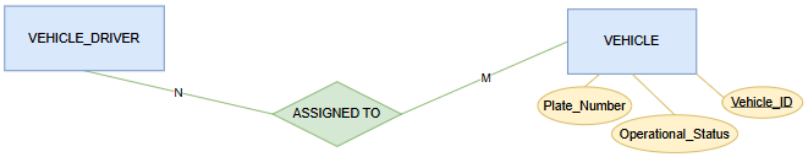

APPOINTMENT

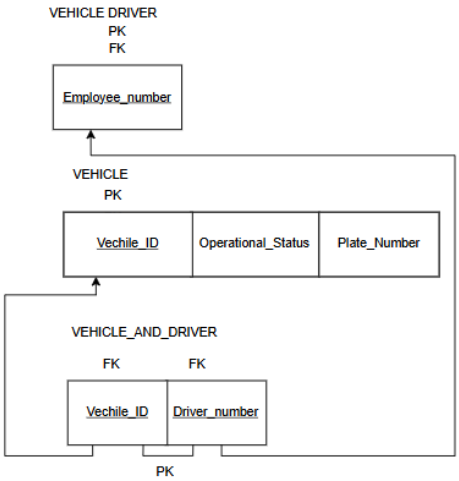
| PK | | | | FK | FK | FK | FK | |
|---|---|---|---|---|---|---|---|---|
| Appointment_id | Status | Date | Time | Patient_number | DEmployee_number | NEmployee_number | treatment_id | Veichle_ID |

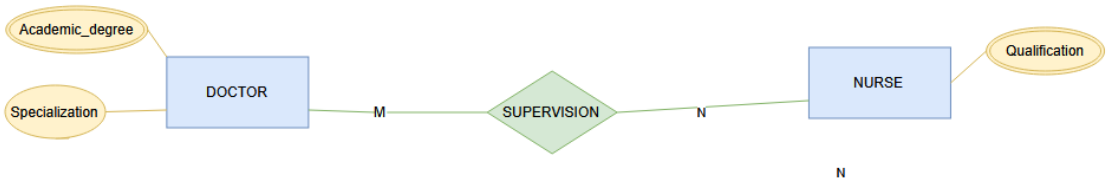
TREATMENT_PLAN

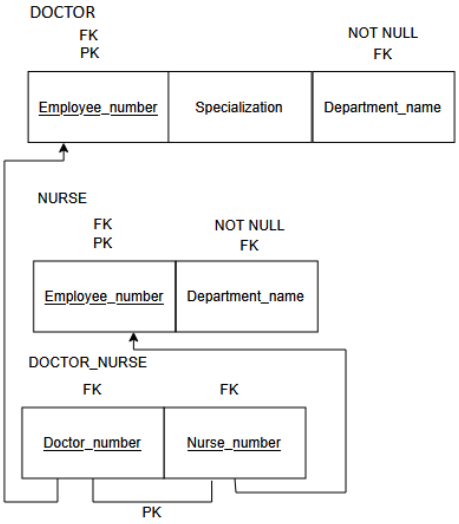
| PK | | | | FK |
|---|---|---|---|---|
| treatment_id | description | Start_date | duration | Dcode |

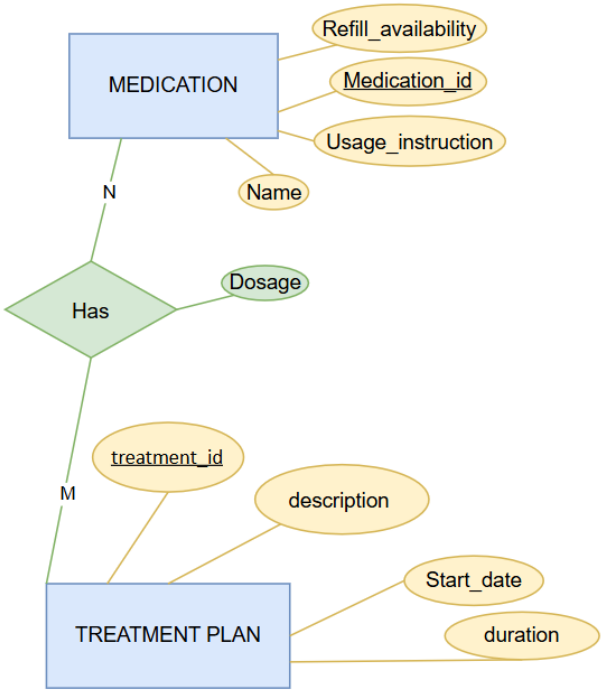
## 3.5 Mapping of binary M-N relationship types

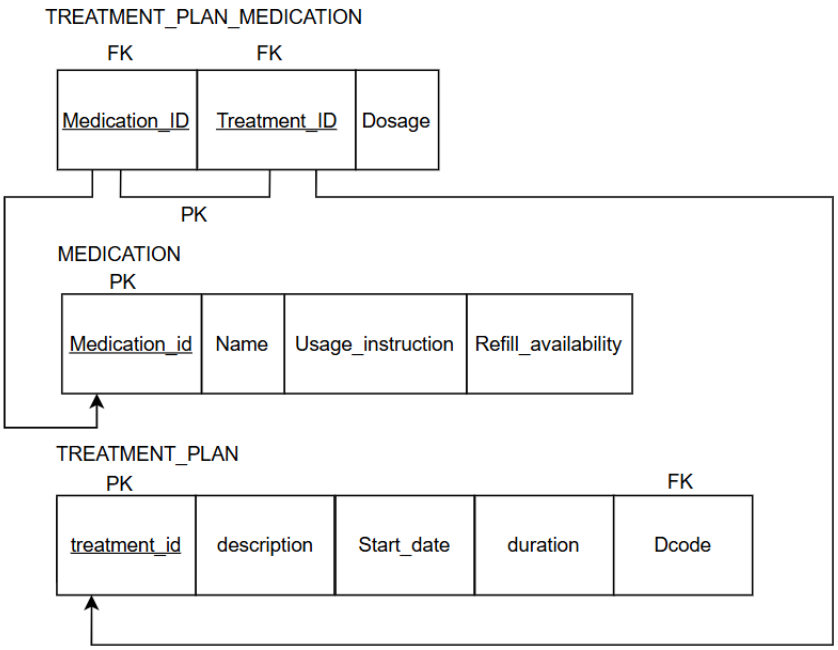


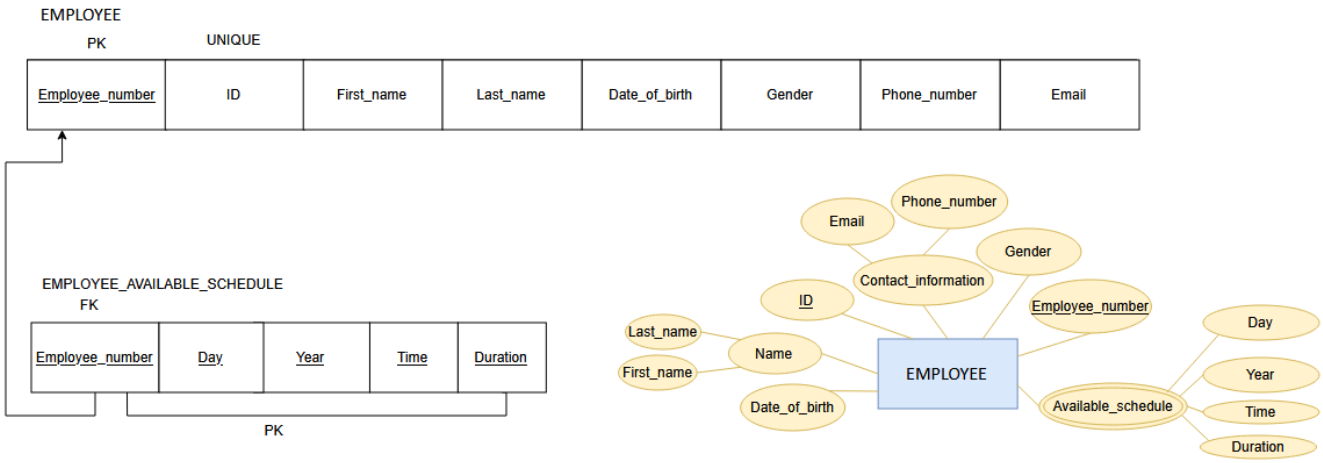



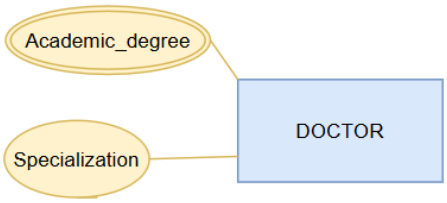



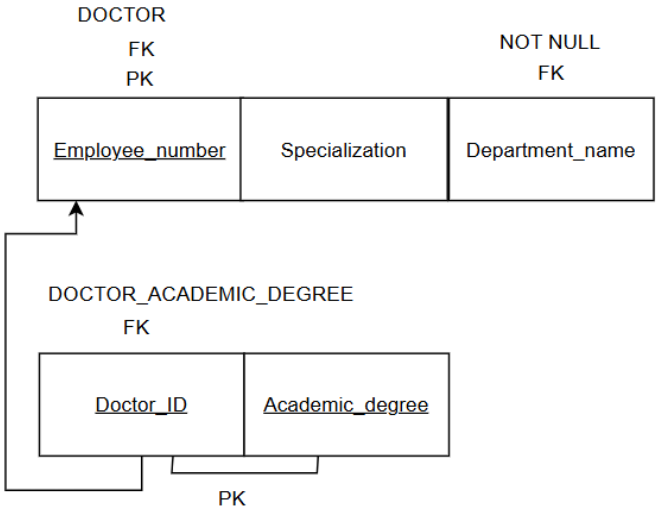

MEDICATION
Refill_availability
Medication_id
Usage_instruction
Name
N
Has
Dosage
M
treatment_id
description
Start_date
duration
TREATMENT PLAN

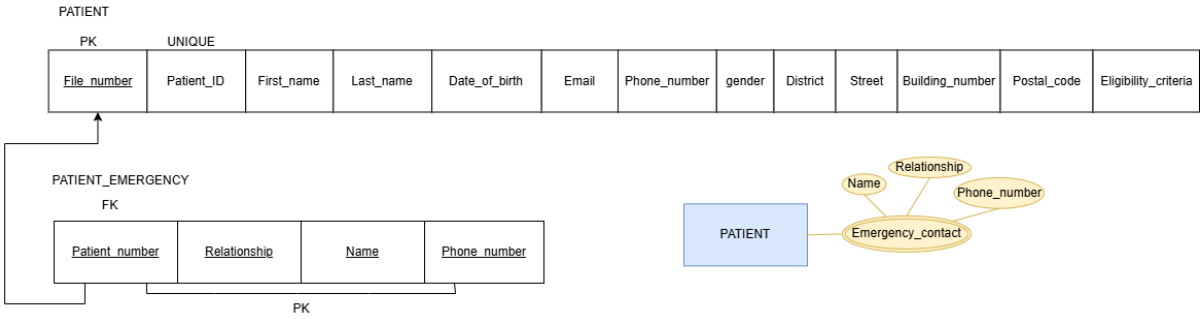

TREATMENT_PLAN_MEDICATION
FK
FK
Medication_ID
Treatment_ID
Dosage
PK
MEDICATION
PK
Medication_id
Name
Usage_instruction
Refill_availability
TREATMENT_PLAN
PK
FK
treatment_id
description
Start_date
duration
Dcode

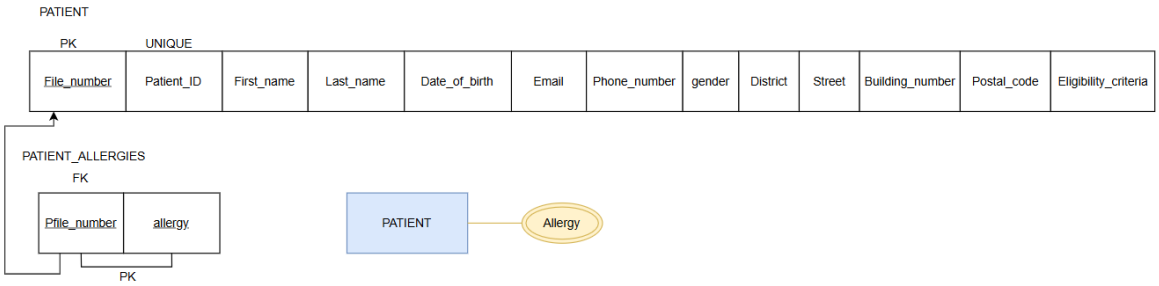
## 3.6 Mapping of multivalued attributes

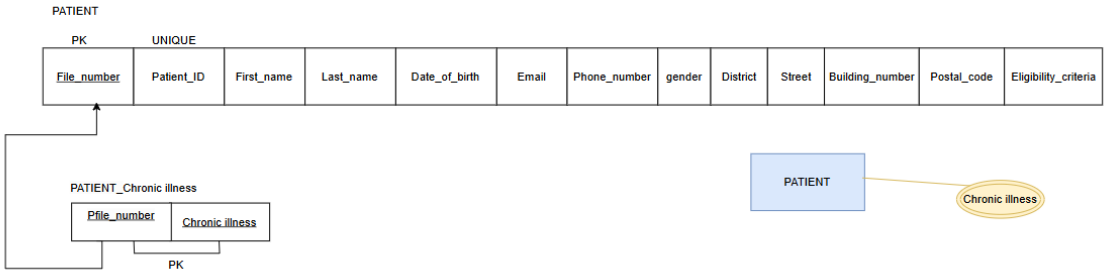


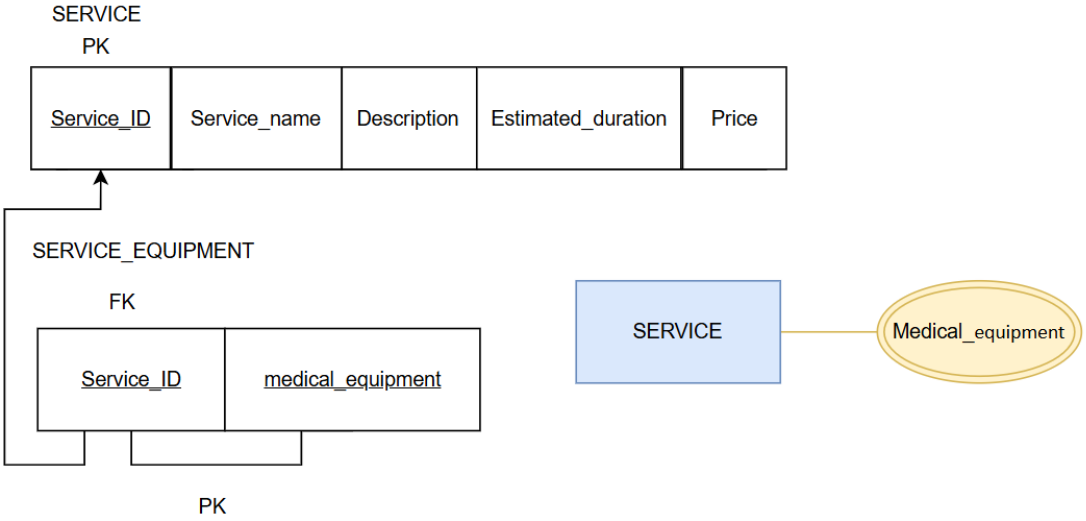



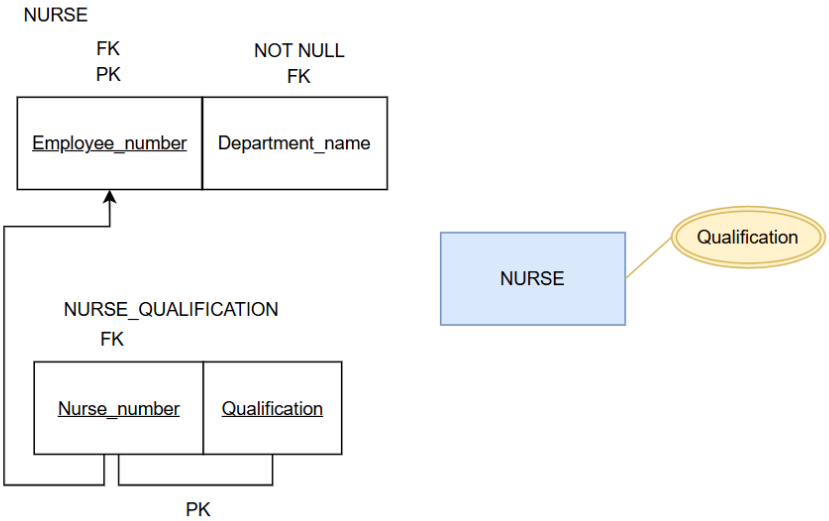




PATIENT
PK
UNIQUE
File_number
Patient_ID
First_name
Last_name
Date_of_birth
Email
Phone_number
gender
District
Street
Building_number
Postal_code
Eligibility_criteria
PATIENT_EMERGENCY
FK
Patient_number
Relationship
Name
Phone_number
PK
PATIENT
Relationship
Name
Phone_number
Emergency_contact


PATIENT
PK
UNIQUE
File_number
Patient_ID
First_name
Last_name
Date_of_birth
Email
Phone_number
gender
District
Street
Building_number
Postal_code
Eligibility_criteria
PATIENT_ALLERGIES
FK
Pfile_number
allergy
PK
PATIENT
Allergy


PATIENT
PK
UNIQUE
File_number
Patient_ID
First_name
Last_name
Date_of_birth
Email
Phone_number
gender
District
Street
Building_number
Postal_code
Eligibility_criteria
PATIENT_Chronic illness
Pfile_number
Chronic illness
PK
PATIENT
Chronic illness



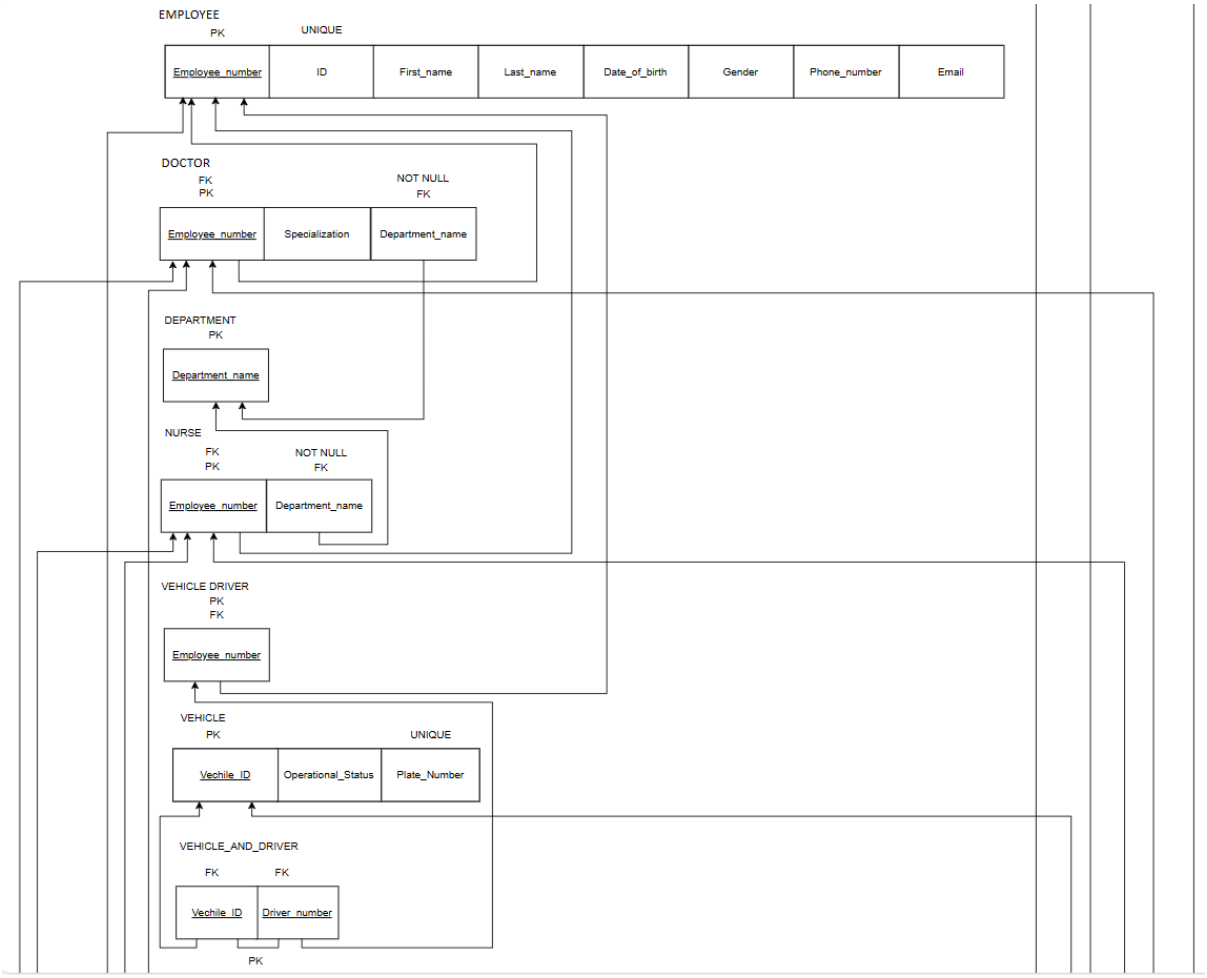



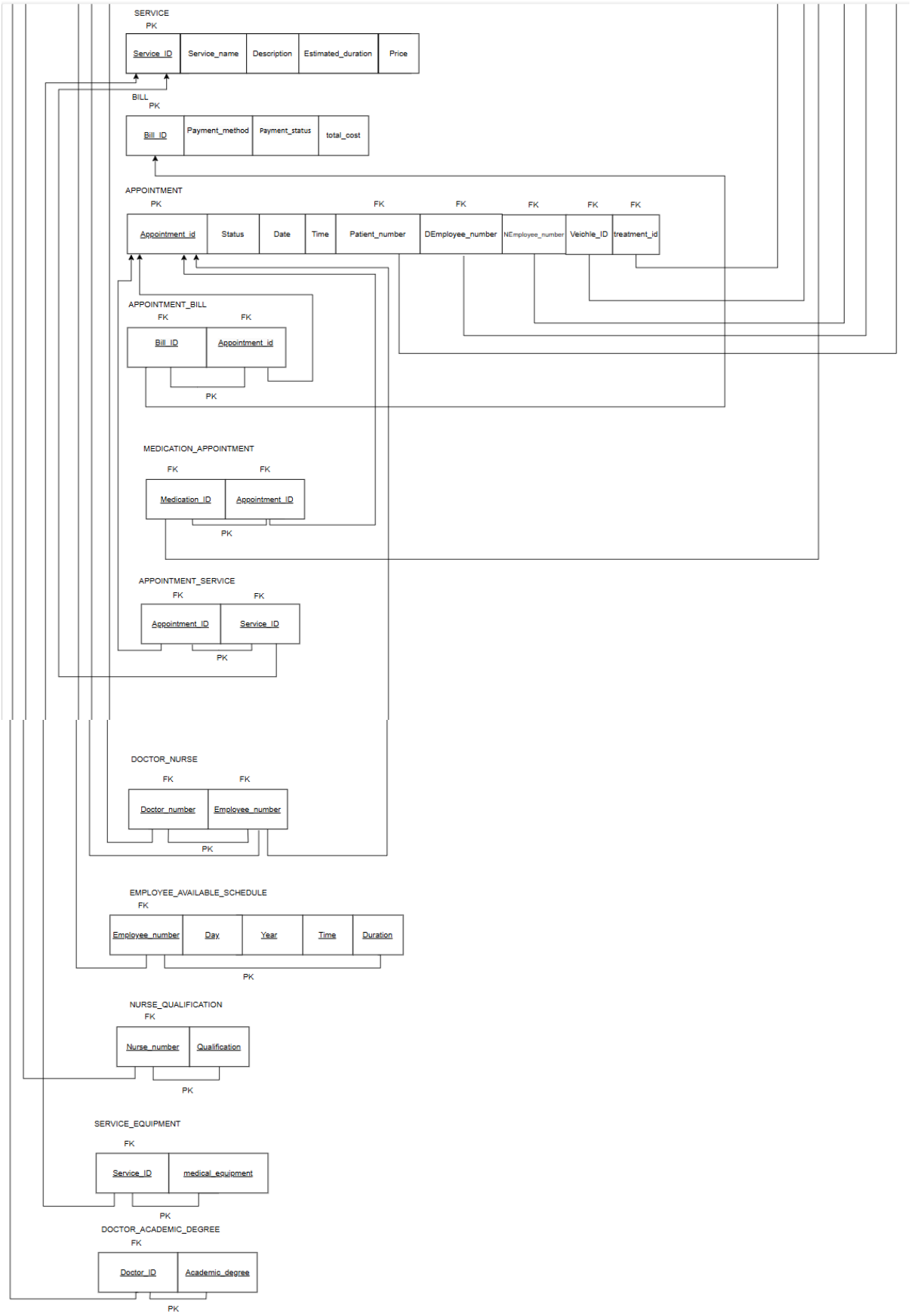

## 3.7 Mapping of N-Ary relationship types

The System does not have any N-Ary entities.

## 3.8 Schema Diagram

STEP 6: Mapping of multivalud attributes


EMPLOYEE
PK
UNIQUE
Employee_number
ID
First_name
Last_name
Date_of_birth
Gender
Phone_number
Email
DOCTOR
FK
PK
NOT NULL
FK
Employee_number
Specialization
Department_name
DEPARTMENT
PK
Department_name
NURSE
FK
PK
NOT NULL
FK
Employee_number
Department_name
VEHICLE DRIVER
PK
FK
Employee_number
VEHICLE
PK
UNIQUE
Vechile_ID
Operational_Status
Plate_Number
VEHICLE_AND_DRIVER
FK
FK
Vechile_ID
Driver_number
PK


SERVICE
PK
Service_ID
Service_name
Description
Estimated_duration
Price
BILL
PK
Bill_ID
Payment_method
Payment_status
total_cost
APPOINTMENT
PK
FK
FK
FK
FK
FK
Appointment_id
Status
Date
Time
Patient_number
DEmployee_number
NEmployee_number
Veichle_ID
treatment_id
APPOINTMENT_BILL
FK
FK
Bill_ID
Appointment_id
PK
MEDICATION_APPOINTMENT
FK
FK
Medication_ID
Appointment_ID
PK
APPOINTMENT_SERVICE
FK
FK
Appointment_ID
Service_ID
PK
DOCTOR_NURSE
FK
FK
Doctor_number
Employee_number
PK
EMPLOYEE_AVAILABLE_SCHEDULE
FK
Employee_number
Day
Year
Time
Duration
PK
NURSE_QUALIFICATION
FK
Nurse_number
Qualification
PK
SERVICE_EQUIPMENT
FK
Service_ID
medical_equipment
PK
DOCTOR_ACADEMIC_DEGREE
FK
Doctor_ID
Academic_degree
PK

# 4 Normalization

## 4.1 First Normal Form

The First normal form(1NF) assigns each attribute to contain only atomic, indivisible values and disallows multivalued attributes and nested relations (attributes whose values for an individual tuple are non-atomic). All tables are in the first normal form because they satisfy the above condition, since our schema design already involves decomposing multivalued attributes into distinct relations.

PATIENT_EMERGENCY

| FK | | | |
|---|---|---|---|
| Patient_number | Relationship | Name | Phone_number |

PK

PATIENT

| PK | UNIQUE | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| File_number | Patient_ID | First_name | Last_name | Date_of_birth | Email | Phone_number | gender | District | Street | Building_number | Postal_code | Eligibility_criteria |

PATIENT_ALLERGIES

| FK | |
|---|---|
| Pfile_number | allergy |

PK

PATIENT_Chronic_illness

| Pfile_number | Chronic illness |
|---|---|

PK

TREATMENT_PLAN_MEDICATION

| FK | FK | |
|---|---|---|
| Medication_ID | Treatment_ID | Dosage |

PK

MEDICATION

| PK | | | |
|---|---|---|---|
| Medication_id | Name | Usage_instruction | Refill_availability |

TREATMENT_PLAN

| PK | | | | FK |
|---|---|---|---|---|
| treatment_id | description | Start_date | duration | code |

DIAGNOSIS

| PK | | |
|---|---|---|
| code | name | clinical_note |

EMPLOYEE

| PK | UNIQUE | | | | | | |
|---|---|---|---|---|---|---|---|
| Employee_number | ID | First_name | Last_name | Date_of_birth | Gender | Phone_number | Email |

DOCTOR

| FK PK | | NOT NULL FK |
|---|---|---|
| Employee_number | Specialization | Department_name |

DEPARTMENT

| PK |
|---|
| Department_name |

NURSE

| FK PK | NOT NULL FK |
|---|---|
| Employee_number | Department_name |

VEHICLE DRIVER

| PK FK |
|---|
| Employee_number |

VEHICLE

| PK | | UNIQUE |
|---|---|---|
| Vechile_ID | Operational_Status | Plate_Number |

VEHICLE_AND_DRIVER

| FK | FK |
|---|---|
| Vechile_ID | Driver_number |

PK

SERVICE

| PK | | | | |
|---|---|---|---|---|
| Service_ID | Service_name | Description | Estimated_duration | Price |

BILL

| PK | | | |
|---|---|---|---|
| Bill_ID | Payment_method | Payment_status | total_cost |

APPOINTMENT

| PK | | | | FK | FK | FK | FK | FK |
|---|---|---|---|---|---|---|---|---|
| Appointment_id | Status | Date | Time | Patient_number | DEmployee_number | NEmployee_number | Veichle_ID | treatment_id |

APPOINTMENT_BILL

| FK | FK |
|---|---|
| Bill_ID | Appointment_id |

PK

MEDICATION_APPOINTMENT

| FK | FK |
|---|---|
| Medication_ID | Appointment_ID |

PK

APPOINTMENT_SERVICE

| FK | FK |
|---|---|
| Appointment_ID | Service_ID |

PK

DOCTOR_NURSE

| FK | FK |
|---|---|
| Employee_number | Employee_number |

PK

EMPLOYEE_AVAILABLE_SCHEDULE

| FK | | | | |
|---|---|---|---|---|
| Employee_number | Day | Year | Time | Duration |

PK

NURSE_QUALIFICATION

| FK | |
|---|---|
| Nurse_number | Qualification |

PK

SERVICE_EQUIPMENT

| FK | |
|---|---|
| Service_ID | medical_equipment |

PK

DOCTOR_ACADEMIC_DEGREE

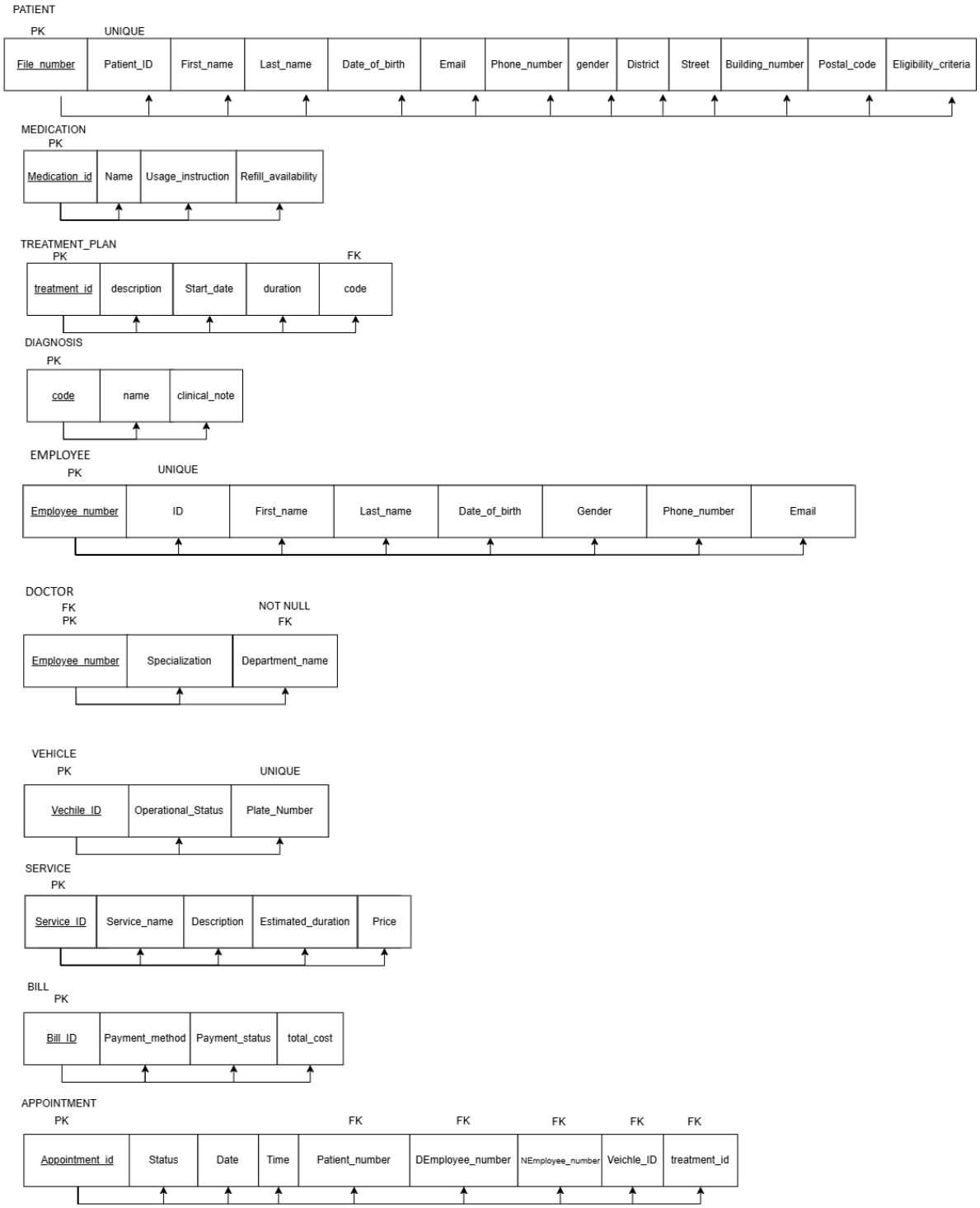
| FK | |
|---|---|
| Doctor_ID | Academic_degree |

PK

Table is in first normal form because there is no multivalued attribute and nested relation.

## 4.2 Second Normal Form

A table in the 2NF should not have partial dependencies, which means that every non-prime attribute should depend on the entire primary key, not just part of it




.In 2NF, there is no possibility for partial dependency because there is only an atomic primary key (one prime attribute).

PATIENT_EMERGENCY

FK

| Patient_number | Relationship | Name | Phone_number |
|---|---|---|---|

PATIENT_ALLERGIES

FK

| Pfile_number | allergy |
|---|---|

PATIENT_Chronic_illness

| Pfile_number | Chronic illness |
|---|---|

NURSE_QUALIFICATION

FK

| Nurse_number | Qualification |
|---|---|

SERVICE_EQUIPMENT

FK

| Service_ID | medical_equipment |
|---|---|

DOCTOR_ACADEMIC_DEGREE

FK

| Doctor_ID | Academic_degree |
|---|---|

DEPARTMENT

PK

| Department_name |
|---|

VEHICLE DRIVER

PK

FK

| Employee_number |
|---|

VEHICLE_AND_DRIVER

| FK | FK |
|---|---|
| Vechile_ID | Driver_number |

APPOINTMENT_BILL

| FK | FK |
|---|---|
| Bill_ID | Appointment_id |

MEDICATION_APPOINTMENT

| FK | FK |
|---|---|
| Medication_ID | Appointment_ID |

APPOINTMENT_SERVICE

| FK | FK |
|---|---|
| Appointment_ID | Service_ID |

DOCTOR_NURSE

| FK | FK |
|---|---|
| Employee_number | Employee_number |

EMPLOYEE_AVAILABLE_SCHEDULE

FK

| Employee_number | Day | Year | Time | Duration |
|---|---|---|---|---|

In 2NF, there is no possibility for partial dependency because there is no non-prime attribute.

TREATMENT_PLAN_MEDICATION

| FK | FK | |
|---|---|---|
| Medication_ID | Treatment_ID | Dosage |

PK

In 2NF, the non-prime attribute is fully dependent on the primary key.

## 4.3 Third Normal Form

3NF prevents any transitive dependencies, transitive dependencies occur when a non-prime attribute depends on another non-prime attribute.

PATIENT_EMERGENCY

FK

| Patient_number | Relationship | Name | Phone_number |
|---|---|---|---|

PATIENT_ALLERGIES

FK

| Pfile_number | allergy |
|---|---|

PATIENT_Chronic_illness

| Pfile_number | Chronic illness |
|---|---|

DEPARTMENT

PK

| Department_name |
|---|

VEHICLE DRIVER

PK

FK

| Employee_number |
|---|

VEHICLE_AND_DRIVER

FK FK

| Vechile_ID | Driver_number |
|---|---|

APPOINTMENT_BILL

FK FK

| Bill_ID | Appointment_id |
|---|---|

MEDICATION_APPOINTMENT

| FK | FK |
|---|---|
| Medication_ID | Appointment_ID |

APPOINTMENT_SERVICE

| FK | FK |
|---|---|
| Appointment_ID | Service_ID |

DOCTOR_NURSE

| FK | FK |
|---|---|
| Employee_number | Employee_number |

EMPLOYEE_AVAILABLE_SCHEDULE

| FK | | | | |
|---|---|---|---|---|
| Employee_number | Day | Year | Time | Duration |

NURSE_QUALIFICATION

| FK | |
|---|---|
| Nurse_number | Qualification |

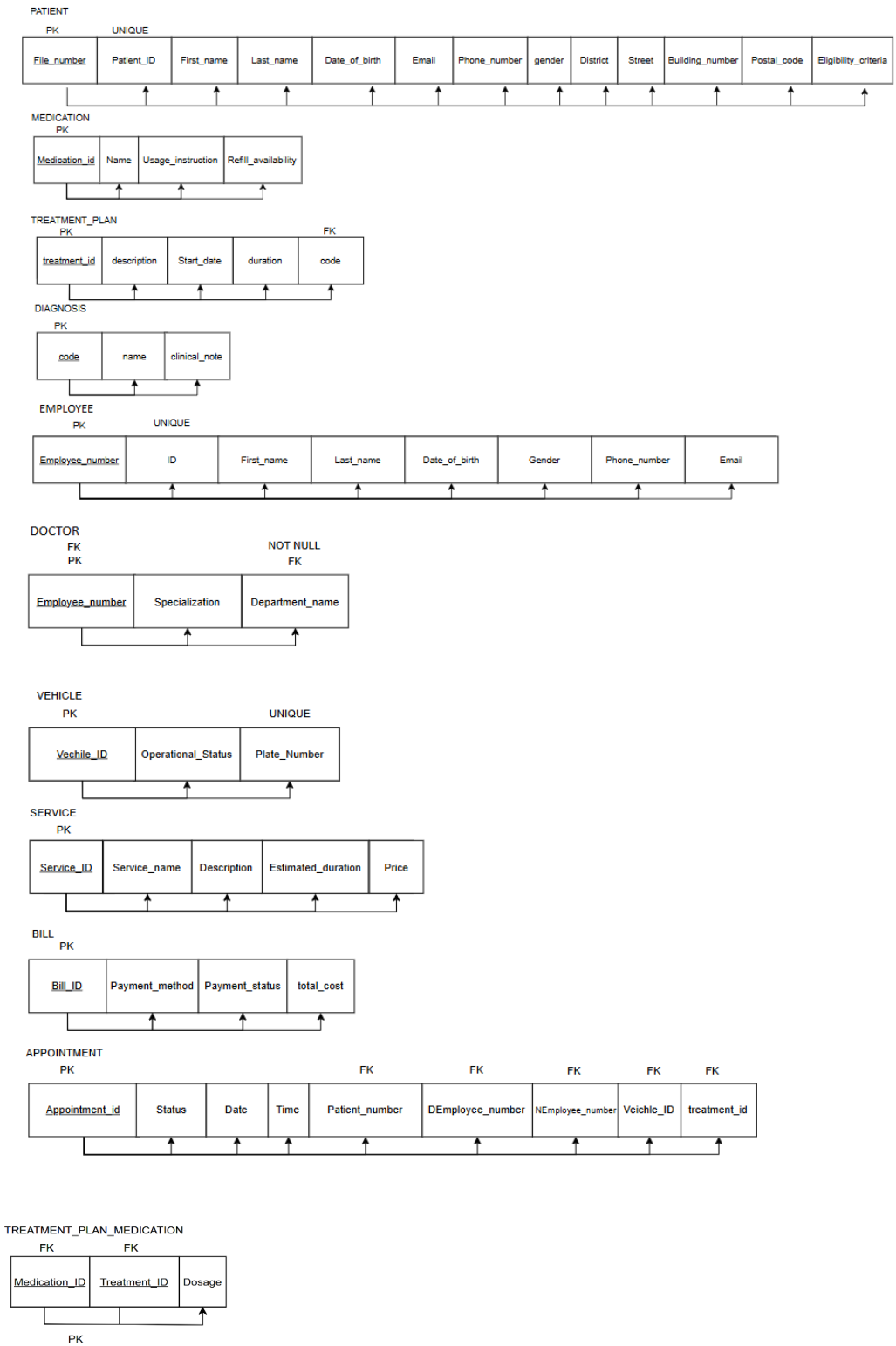
SERVICE_EQUIPMENT

| FK | |
|---|---|
| Service_ID | medical_equipment |

DOCTOR_ACADEMIC_DEGREE

| FK | |
|---|---|
| Doctor_ID | Academic_degree |

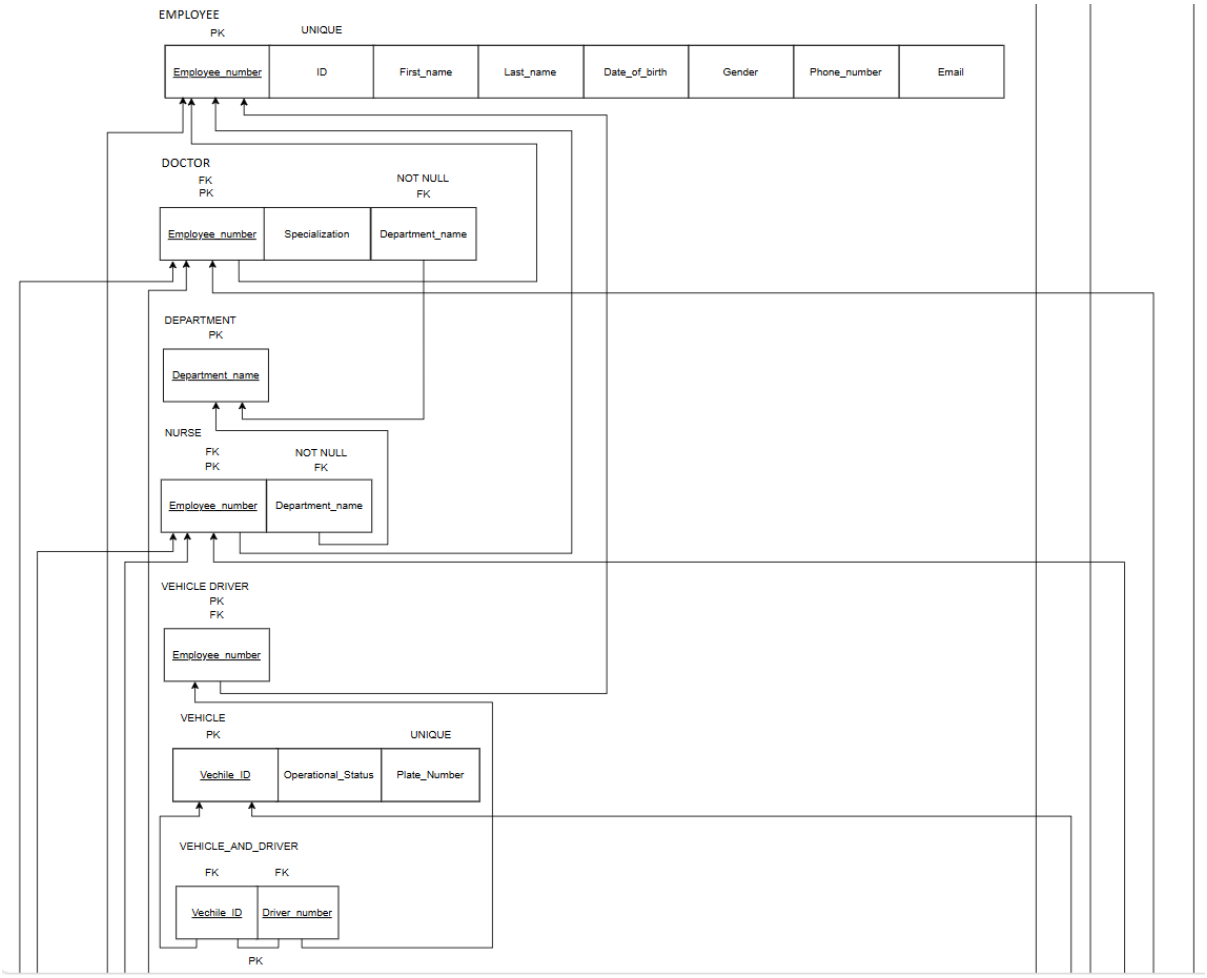
In 3NF, there are no non-prime attributes, so there is no possibility of transitive dependency.

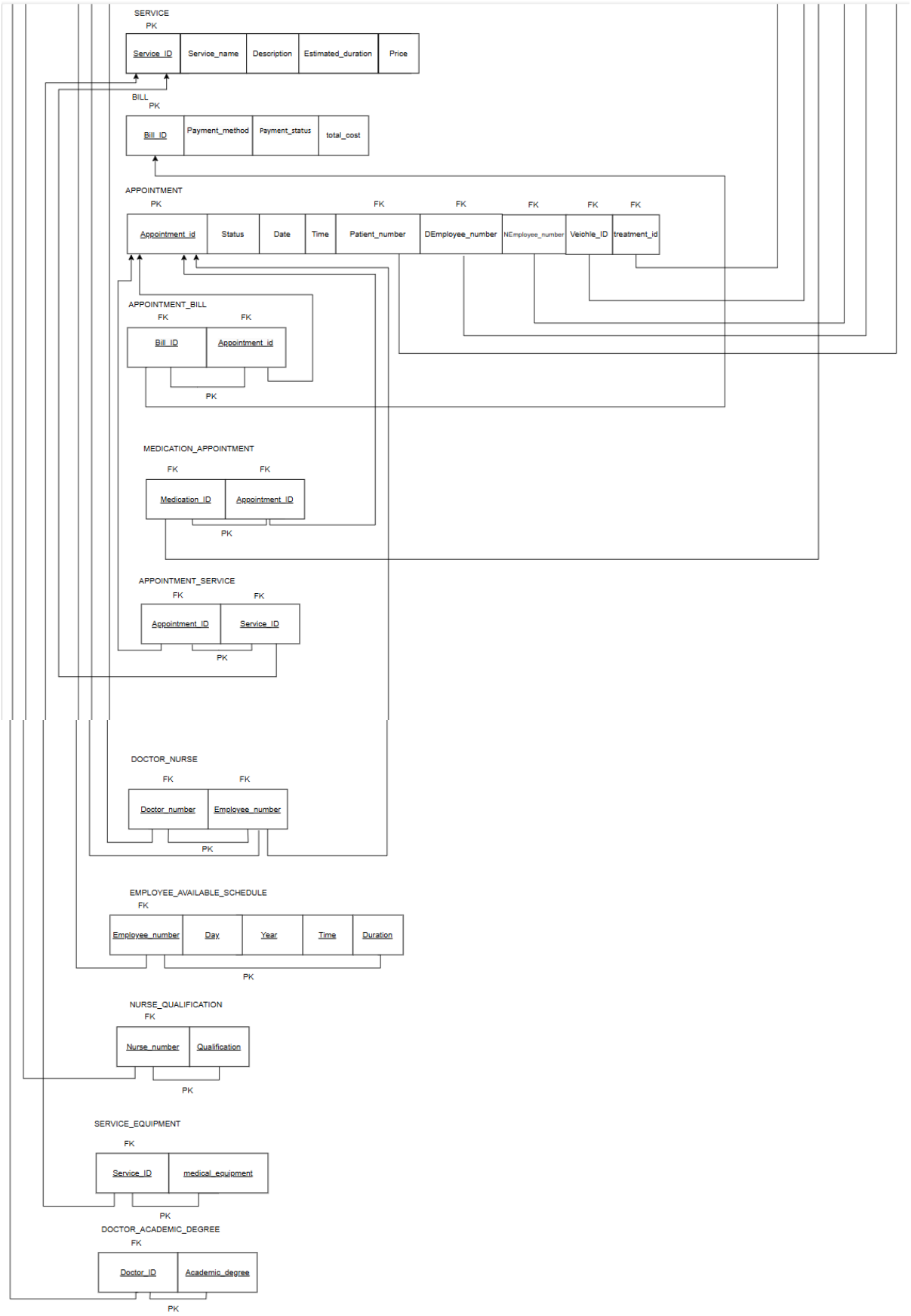



In 3NF, all attributes depend on the key, and there are no transitive dependencies.

# 5 Final DB Schema Diagram

STEP 6: Mapping of multivalud attributes


EMPLOYEE
PK
UNIQUE
Employee_number
ID
First_name
Last_name
Date_of_birth
Gender
Phone_number
Email
DOCTOR
FK
PK
NOT NULL
FK
Employee_number
Specialization
Department_name
DEPARTMENT
PK
Department_name
NURSE
FK
PK
NOT NULL
FK
Employee_number
Department_name
VEHICLE DRIVER
PK
FK
Employee_number
VEHICLE
PK
UNIQUE
Vechile_ID
Operational_Status
Plate_Number
VEHICLE_AND_DRIVER
FK
FK
Vechile_ID
Driver_number
PK


SERVICE
PK
Service_ID
Service_name
Description
Estimated_duration
Price
BILL
PK
Bill_ID
Payment_method
Payment_status
total_cost
APPOINTMENT
PK
FK
FK
FK
FK
FK
Appointment_id
Status
Date
Time
Patient_number
DEmployee_number
NEmployee_number
Veichle_ID
treatment_id
APPOINTMENT_BILL
FK
FK
Bill_ID
Appointment_id
PK
MEDICATION_APPOINTMENT
FK
FK
Medication_ID
Appointment_ID
PK
APPOINTMENT_SERVICE
FK
FK
Appointment_ID
Service_ID
PK
DOCTOR_NURSE
FK
FK
Doctor_number
Employee_number
PK
EMPLOYEE_AVAILABLE_SCHEDULE
FK
Employee_number
Day
Year
Time
Duration
PK
NURSE_QUALIFICATION
FK
Nurse_number
Qualification
PK
SERVICE_EQUIPMENT
FK
Service_ID
medical_equipment
PK
DOCTOR_ACADEMIC_DEGREE
FK
Doctor_ID
Academic_degree
PK

# PART III: IMPLEMENTATION

## 6 Table Creation Script

### 6.1 <*DEPARTMENT* > TABLE

CREATE TABLE Department(

Dname VARCHAR2(100) ,

CONSTRAINT pk_department PRIMARY KEY(dname)

);

```
SQL> CREATE TABLE DEPARTMENT(
  2  Dname VARCHAR2(100) ,
  3  CONSTRAINT pk_department PRIMARY KEY(dname)
  4  );

Table created.
```

### 6.2 <*EMPLOYEE* > TABLE

CREATE TABLE EMPLOYEE (

Employee_number NUMBER(6) , ID VARCHAR2(10) UNIQUE

NOT NULL, First_name VARCHAR2(50) NOT NULL, Last_name VARCHAR2(50) NOT NULL,

Date_of_birth DATE NOT NULL, Gender CHAR(1) CHECK (Gender IN ('M', 'F')),

Phone_number VARCHAR2(10), Email VARCHAR2(100), CONSTRAINT pk_employee

PRIMARY KEY(Employee_number) );

```
SQL> CREATE TABLE EMPLOYEE ( Employee_number NUMBER(6) , ID VARCHAR2(10) UNIQUE
  2  NOT NULL, First_name VARCHAR2(50) NOT NULL, Last_name VARCHAR2(50) NOT NULL,
  3  Date_of_birth DATE NOT NULL, Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
  4  Phone_number VARCHAR2(10), Email VARCHAR2(100), CONSTRAINT pk_employee
  5  PRIMARY KEY(Employee_number) );

Table created.
```

## 6.3 <*PATIENT* > TABLE

CREATE TABLE PATIENT(

File_number NUMBER (7) , Patient_ID NUMBER (10) UNIQUE,

First_name VARCHAR2(50) NOT NULL, Last_name VARCHAR2(50) NOT NULL,

Date_of_birth DATE NOT NULL, Email VARCHAR2(100), Phone_number VARCHAR2(10),

Gender VARCHAR2(1) CHECK (Gender IN ('M', 'F')), District VARCHAR2(25), Streat

VARCHAR2(50), Building_number VARCHAR2(25), Postal_code VARCHAR2(10),

Eligibility_criteria VARCHAR2(255) NOT NULL, CONSTRAINT pk_patient PRIMARY KEY

(file_number) );

```
SQL> CREATE TABLE PATIENT( File_number NUMBER (7) , Patient_ID NUMBER (10) UNIQUE,
  2  First_name VARCHAR2(50) NOT NULL, Last_name VARCHAR2(50) NOT NULL,
  3  Date_of_birth DATE NOT NULL, Email VARCHAR2(100), Phone_number VARCHAR2(10),
  4  Gender VARCHAR2(1) CHECK (Gender IN ('M', 'F')), District VARCHAR2(25), Streat
  5  VARCHAR2(50), Building_number VARCHAR2(25), Postal_code VARCHAR2(10),
  6  Eligibility_criteria VARCHAR2(255) NOT NULL, CONSTRAINT pk_patient PRIMARY KEY
  7  (file_number) );

Table created.
```

## 6.4 < *VEHICLE*> TABLE

CREATE TABLE VEHICLE(

VehicleID NUMBER(10) ,

OperationalStatus VARCHAR2(50),

PlateNumber NUMBER(10) UNIQUE,

CONSTRAINT pk_vehicle PRIMARY KEY(VehicleID),

);

```
SQL> CREATE TABLE VEHICLE(
  2  VehicleID NUMBER(10) ,
  3  OperationalStatus VARCHAR2(50),
  4  PlateNumber NUMBER(10) UNIQUE,
  5  CONSTRAINT pk_vehicle PRIMARY KEY(VehicleID)
  6  );

Table created.
```

## 6.5 <*MEDICATION* > TABLE

CREATE TABLE MEDICATION (

Medication_id NUMBER(10),

Name VARCHAR2(100),

Usage_instruction VARCHAR2(500),

Refill_availability CHAR(1), --Use 'Y'/'N' or '1'/'0' to simulate BOOLEAN,

CONSTRAINT pk_medication PRIMARY KEY(Medication_id)

);

```
SQL> CREATE TABLE MEDICATION (
  2  Medication_id NUMBER(10),
  3  Name VARCHAR2(100),
  4  Usage_instruction VARCHAR2(500),
  5  Refill_availability CHAR(1),
  6  CONSTRAINT pk_MEDICATION PRIMARY KEY(Medication_id)
  7  );

Table created.
```

## 6.6 <*DIAGNOSIS*> TABLE

CREATE TABLE DIAGNOSIS (

Code VARCHAR2(10),

Name VARCHAR2(100) NOT NULL,

Clinical_note VARCHAR2(500),

CONSTRAINT pk_diagnosis PRIMARY KEY(code)

);

```
SQL> CREATE TABLE DIAGNOSIS (
  2  Code VARCHAR2(10),
  3  Name VARCHAR2(100) NOT NULL,
  4  Clinical_note  VARCHAR2(500),
  5  CONSTRAINT pk_diagnosis PRIMARY KEY(Code)
  6  );

Table created.
```

## 6.7 <*SERVICE*> TABLE

CREATE TABLE SERVICE (

service_id NUMBER(10),

name VARCHAR2(100) NOT NULL,

description VARCHAR2(255),

estimated_duration NUMBER(3) ,

price NUMBER(8,2) ,

CONSTRAINT pk_service PRIMARY KEY(service_id)

);

```
SQL> CREATE TABLE SERVICE (
  2  service_id NUMBER(10),
  3  name VARCHAR2(100) NOT NULL,
  4  description VARCHAR2(255),
  5  estimated_duration NUMBER(3),
  6  price NUMBER(8,2),
  7  CONSTRAINT pk_service PRIMARY KEY(service_id)
  8  );

Table created.
```

## 6.8 <*BILL*> TABLE

CREATE TABLE BILL(

bill_id NUMBER(10),

payment_method VARCHAR2(50) ,

payment_status VARCHAR2(50),

total_cost NUMBER(10,2),

CONSTRAINT pk_bill PRIMARY KEY(bill_id)

);

```
SQL> CREATE TABLE BILL(
  2  bill_id NUMBER(10),
  3  payment_method VARCHAR2(50),
  4  payment_status VARCHAR2(50),
  5  total_cost NUMBER(10,2),
  6  CONSTRAINT pk_bill PRIMARY KEY(bill_id)
  7  );

Table created.
```

## 6.9 <*DOCTOR*> TABLE

CREATE TABLE DOCTOR(

Employee_number NUMBER(6),

Specialization VARCHAR2(100),

CONSTRAINT pk_doctor PRIMARY KEY(Employee_number),

CONSTRAINT fk_doctor_employee FOREIGN KEY (Employee_number)

REFERENCES EMPLOYEE(Employee_number)

ON DELETE CASCADE

);

```
SQL> CREATE TABLE DOCTOR(
  2  Employee_number NUMBER(6),
  3  Specialization VARCHAR2(100),
  4  CONSTRAINT pk_doctor PRIMARY KEY(Employee_number),
  5  CONSTRAINT fk_doctor_employee FOREIGN KEY (Employee_number)
  6  REFERENCES EMPLOYEE(Employee_number)
  7  ON DELETE CASCADE
  8  );

Table created.
```

## 6.10 <*NURSE*> TABLE

CREATE TABLE NURSE(

Enumber NUMBER (6),

Dname VARCHAR2 (100) NOT NULL,

CONSTRAINT pk_nurse PRIMARY KEY Enumber (),

CONSTRAINT fk_nurse_employee FOREIGN KEY (Enumber) REFERENCES

Employee(employee_Number) ON DELETE CASCADE,

CONSTRAINT fk_nurse_department FOREIGN KEY (Dname) REFERENCES

Department(Dname) ON DELETE CASCADE

);

```
SQL> CREATE TABLE NURSE(
  2  Enumber NUMBER (6),
  3  Dname VARCHAR2 (100) NOT NULL,
  4  CONSTRAINT pk_nurse PRIMARY KEY(Enumber),
  5  CONSTRAINT fk_nurse_employee FOREIGN KEY (Enumber) REFERENCES
  6  Employee(employee_Number) ON DELETE CASCADE,
  7  CONSTRAINT fk_nurse_department FOREIGN KEY (Dname) REFERENCES
  8  Department(Dname) ON DELETE CASCADE
  9  );

Table created.
```

## 6.11 <*VEHICLE_DRIVER*> TABLE

CREATE TABLE VEHICLE_DRIVER(

Enumber NUMBER(6) ,

CONSTRAINT pk_vehicle_driver PRIMARY KEY (Enumber),

CONSTRAINT fk_vehicle_employee FOREIGN KEY (Enumber) REFERENCES

Employee(employee_Number) ON DELETE CASCADE

);

```
SQL> CREATE TABLE VEHICLE_DRIVER(
  2  Enumber NUMBER(6),
  3  CONSTRAINT pk_vehicle_driver PRIMARY KEY(Enumber),
  4  CONSTRAINT fk_vehicle_employee FOREIGN KEY (Enumber) REFERENCES
  5  Employee(employee_Number) ON DELETE CASCADE
  6  );

Table created.
```

## 6.12 <*TREATMENT_PLAN*> TABLE

CREATE TABLE TREATMENT_PLAN (

Treatment_id NUMBER(10),

Description VARCHAR2(500),

Start_date DATE NOT NULL,

Duration NUMBER(3) CHECK (Duration > 0),

Dcode VARCHAR2(10),
CONSTRAINT pk_treatment PRIMARY KEY(Treatment_id),

CONSTRAINT fk_treatment_diagnosis FOREIGN KEY (Dcode)

REFERENCES DIAGNOSIS(Code)

ON DELETE SET NULL

);

```
SQL> CREATE TABLE TREATMENT_PLAN (
  2  Treatment_id NUMBER(10),
  3  Description VARCHAR2(500),
  4  Start_date DATE NOT NULL,
  5  Duration NUMBER(3) CHECK (Duration > 0),
  6  Dcode VARCHAR2(10),
  7  CONSTRAINT pk_treatment PRIMARY KEY(Treatment_id),
  8  CONSTRAINT fk_treatment_diagnosis FOREIGN KEY (Dcode)
  9  REFERENCES DIAGNOSIS(Code)
 10  ON DELETE SET NULL
 11  );

Table created.
```

## 6.13 < *PATIENT_EMERGENCY* > TABLE

CREATE TABLE PATIENT_EMERGENCY(

Patient_number NUMBER (7) ,

Relationship VARCHAR2(50),

Name VARCHAR2(50),

Phone_number VARCHAR2(20) NOT NULL,

CONSTRAINT fk_patient_emergency FOREIGN KEY (patient_number) REFERENCES PATIENT(file_number) ON DELETE CASCADE,

CONSTRAINT pk_patient_emergency PRIMARY KEY(patient_number, relationship, name, phone_number));

```
SQL> CREATE TABLE PATIENT_EMERGENCY(
  2  Patient_number NUMBER (7) ,
  3  Relationship VARCHAR2(50),
  4  Name VARCHAR2(50),
  5  Phone_number VARCHAR2(20) NOT NULL,
  6  CONSTRAINT fk_patient_emergency FOREIGN KEY (patient_number)
  7  REFERENCES PATIENT(file_number)
  8  ON DELETE CASCADE,
  9  CONSTRAINT pk_patient_emergency PRIMARY KEY(patient_number, relationship, name,
 10  phone_number));

Table created.
```

## 6.14 <*PATIENT_CHRONIC_ILLNESS*> TABLE

CREATE TABLE PATIENT_CHRONIC_ILLNESS(

Pfile_number NUMBER (7),

Chronic_illness VARCHAR2(255),

CONSTRAINT fk_patient_chronic_illness FOREIGN KEY (pfile_number) REFERENCES PATIENT(file_number) ON DELETE CASCADE,

CONSTRAINT pk_patient_chronic PRIMARY KEY(pfile_number, chronic_illness));

```
SQL> CREATE TABLE PATIENT_CHRONIC_ILLNESS(
  2  Pfile_number NUMBER (7),
  3  Chronic_illness VARCHAR2(255),
  4  CONSTRAINT fk_patient_chronic_illness FOREIGN KEY (pfile_number)
  5  REFERENCES PATIENT(file_number)
  6  ON DELETE CASCADE,
  7  CONSTRAINT pk_patient_chronic PRIMARY KEY(pfile_number, chronic_illness));

Table created.
```

## 6.15 <*VEHICLE_AND_DRIVER*> TABLE

CREATE TABLE VEHICLE_AND_DRIVER(

VehicleID NUMBER(10),

Dnumber NUMBER(10),

CONSTRAINT pk_vehicle_and_driver PRIMARY KEY (VehicleID, Dnumber),

CONSTRAINT fk_vehicle_driver FOREIGN KEY (VehicleID) REFERENCES Vehicle (VehicleID) ON DELETE CASCADE,

CONSTRAINT fk_driver_employee FOREIGN KEY (Dnumber) REFERENCES Vehicle_driver(Enumber) ON DELETE CASCADE );

```
SQL> CREATE TABLE VEHICLE_AND_DRIVER(
  2  VehicleID NUMBER(10),
  3  Dnumber NUMBER(10),
  4  CONSTRAINT pk_vehicle_and_driver PRIMARY KEY (VehicleID, Dnumber),
  5  CONSTRAINT fk_vehicle_driver FOREIGN KEY (VehicleID) REFERENCES Vehicle (VehicleID)
  6  ON DELETE CASCADE,
  7  CONSTRAINT fk_driver_employee FOREIGN KEY (Dnumber) REFERENCES
  8  Vehicle_driver(Enumber) ON DELETE CASCADE
  9  );

Table created.
```

## 6.16 < *TREATMENT_PLAN_MEDICATION* > TABLE

CREATE TABLE TREATMENT_PLAN_MEDICATION(

Medication_ID NUMBER(10),

Treatment_ID NUMBER(10),

Dosage VARCHAR2(255),

CONSTRAINT fk_treat_med_medicationid FOREIGN KEY(Medication_ID) REFERENCES MEDICATION(Medication_ID) ON DELETE CASCADE,

CONSTRAINT fk_treat_med_treatmentid FOREIGN KEY (Treatment_ID) REFERENCES TREATMENT_PLAN (Treatment_ID) ON DELETE CASCADE,

CONSTRAINT pk_treat_med_medication PRIMARY KEY(medication_id, treatment_id) );

```
SQL> CREATE TABLE TREATMENT_PLAN_MEDICATION(
  2  Medication_ID NUMBER(10),
  3  Treatment_ID NUMBER(10),
  4  Dosage VARCHAR2(255),
  5  CONSTRAINT fk_treat_med_medicationid FOREIGN KEY(Medication_ID)
  6  REFERENCES MEDICATION(Medication_ID) ON DELETE CASCADE,
  7  CONSTRAINT fk_treat_med_treatmentid FOREIGN KEY
  8  (Treatment_ID)
  9  REFERENCES TREATMENT_PLAN (Treatment_ID)
 10  ON DELETE CASCADE,
 11  CONSTRAINT pk_treat_med_medication PRIMARY KEY(medication_id,
 12  treatment_id)
 13  );

Table created.
```

## 6.17 < *EMPLOYEE_AVAILABLE_SCHEDULE*> TABLE

CREATE TABLE EMPLOYEE_AVAILABLE_SCHEDULE (

Employee_number NUMBER(6),

Day VARCHAR2(10), Year NUMBER(4),

Time TIMESTAMP,

Duration NUMBER(2),

CONSTRAINT pk_employee_schedule PRIMARY KEY (Employee_number, Day, Year, Time), CONSTRAINT fk_employee_schedule_number FOREIGN KEY (Employee_number) REFERENCES EMPLOYEE(Employee_number) );

```
SQL> CREATE TABLE EMPLOYEE_AVAILABLE_SCHEDULE (
  2  Employee_number NUMBER(6),
  3  Day VARCHAR2(10),
  4  Year NUMBER(4),
  5  Time TIMESTAMP,
  6  Duration NUMBER(2),
  7  CONSTRAINT pk_employee_schedule PRIMARY KEY (Employee_number, Day,
  8  Year, Time),
  9  CONSTRAINT fk_employee_schedule_number FOREIGN KEY
 10  (Employee_number) REFERENCES EMPLOYEE(Employee_number)
 11  );

Table created.
```

## 6.18 <*DOCTOR_NURSE*> TABLE

CREATE TABLE DOCTOR_NURSE (

Doctor_number NUMBER(6),

Nurse_number NUMBER(6),

CONSTRAINT PK_DOCTOR_NURSE PRIMARY KEY (Doctor_number, Nurse_number),

CONSTRAINT FK_DOC_NURSE_DOCTOR FOREIGN KEY (Doctor_number) REFERENCES EMPLOYEE(Employee_number),

CONSTRAINT FK_DOC_NURSE_NURSE FOREIGN KEY (Nurse_number) REFERENCES EMPLOYEE(Employee_number) );

```
SQL> CREATE TABLE DOCTOR_NURSE (
  2  Doctor_number NUMBER(6),
  3  Nurse_number NUMBER(6),
  4  CONSTRAINT PK_DOCTOR_NURSE
  5  PRIMARY KEY (Doctor_number, Nurse_number),
  6  CONSTRAINT FK_DOC_NURSE_DOCTOR
  7  FOREIGN KEY (Doctor_number)
  8  REFERENCES EMPLOYEE(Employee_number),
  9  CONSTRAINT FK_DOC_NURSE_NURSE
 10  FOREIGN KEY (Nurse_number)
 11  REFERENCES EMPLOYEE(Employee_number)
 12  );

Table created.
```

## 6.19 < *NURSE_QUALIFICATION* > TABLE

CREATE TABLE NURSE_QUALIFICATION (

Nurse_number NUMBER(6),

Qualification VARCHAR2(100),

CONSTRAINT PK_NURSE_QUALIFICATION PRIMARY KEY (Nurse_number, Qualification), CONSTRAINT FK_NURSE_QUAL_EMPLOYEE FOREIGN KEY (Nurse_number) REFERENCES EMPLOYEE(Employee_number) );

```
SQL> CREATE TABLE NURSE_QUALIFICATION (
  2  Nurse_number NUMBER(6),
  3  Qualification VARCHAR2(100),
  4  CONSTRAINT PK_NURSE_QUALIFICATION
  5  PRIMARY KEY (Nurse_number, Qualification),
  6  CONSTRAINT FK_NURSE_QUAL_EMPLOYEE
  7  FOREIGN KEY (Nurse_number)
  8  REFERENCES EMPLOYEE(Employee_number)
  9  );

Table created.
```

## 6.20 < *SERVICE_EQUIPMENT* > TABLE

CREATE TABLE SERVICE_EQUIPMENT (

Service_ID NUMBER(10),

medical_equipment VARCHAR2(100),

CONSTRAINT PK_SERVICE_EQUIPMENT PRIMARY KEY (Service_ID, medical_equipment),
CONSTRAINT FK_SVC_EQUIP_SERVICE FOREIGN KEY (Service_ID) REFERENCES SERVICE(Service_ID) );

```
SQL> CREATE TABLE SERVICE_EQUIPMENT (
  2  Service_ID NUMBER(10),
  3  medical_equipment VARCHAR2(100),
  4  CONSTRAINT PK_SERVICE_EQUIPMENT
  5  PRIMARY KEY (Service_ID, medical_equipment),
  6  CONSTRAINT FK_SVC_EQUIP_SERVICE
  7  FOREIGN KEY (Service_ID)
  8  REFERENCES SERVICE(Service_ID)
  9  );

Table created.
```

## 6.21 < *DOCTOR_ACADEMIC_DEGRE* > TABLE

CREATE TABLE DOCTOR_ACADEMIC_DEGREE(

Doctor_ID NUMBER(6),

Academic_degree VARCHAR2(100),

CONSTRAINT PK_DOCTOR_ACADEMIC_DEGREE PRIMARY KEY (Doctor_ID, Academic_degree),
CONSTRAINT FK_DOC_DEGREE_EMPLOYEE FOREIGN KEY (Doctor_ID) REFERENCES EMPLOYEE(Employee_number) );

```
SQL> CREATE TABLE DOCTOR_ACADEMIC_DEGREE(
  2  Doctor_ID NUMBER(6),
  3  Academic_degree VARCHAR2(100),
  4  CONSTRAINT PK_DOCTOR_ACADEMIC_DEGREE
  5  PRIMARY KEY (Doctor_ID, Academic_degree),
  6  CONSTRAINT FK_DOC_DEGREE_EMPLOYEE
  7  FOREIGN KEY (Doctor_ID)
  8  REFERENCES EMPLOYEE(Employee_number)
  9  );

Table created.
```

## 6.22 <*APPOINTMENT* > TABLE

CREATE TABLE appointment(

appointment_id NUMBER(10) PRIMARY KEY,

status VARCHAR2(50),

appointment_date DATE,

appointment_time TIMESTAMP,

patient_number NUMBER(7),

DEmployee_number NUMBER(6),

NEmployee_number NUMBER(6),

vehicle_id NUMBER(10),

treatment_id NUMBER(10),

CONSTRAINT fk_appt_patient FOREIGN KEY(patient_number) REFERENCES patient(file_number) ON DELETE CASCADE,

CONSTRAINT fk_appt_doctor FOREIGN KEY(DEmployee_number) REFERENCES doctor(Employee_number) ON DELETE SET NULL,

CONSTRAINT fk_appt_nurse FOREIGN KEY(NEmployee_number) REFERENCES nurse(Enumber) ON DELETE SET NULL, CONSTRAINT fk_appt_vehicle FOREIGN KEY(vehicle_id) REFERENCES vehicle(VehicleID) ON DELETE SET NULL,

CONSTRAINT fk_appt_treatment FOREIGN KEY(treatment_id) REFERENCES treatment_plan(Treatment_id) ON DELETE SET NULL);

```
SQL> CREATE TABLE appointment(
  2  appointment_id NUMBER(10) PRIMARY KEY,
  3  status VARCHAR2(50),
  4  appointment_date DATE,
  5  appointment_time TIMESTAMP,
  6  patient_number NUMBER(7),
  7  DEmployee_number NUMBER(6),
  8  NEmployee_number NUMBER(6),
  9  vehicle_id NUMBER(10),
 10  treatment_id NUMBER(10),
 11  CONSTRAINT fk_appt_patient FOREIGN KEY(patient_number) REFERENCES patient(file_number) ON DELETE CASCADE,
 12  CONSTRAINT fk_appt_doctor FOREIGN KEY(DEmployee_number) REFERENCES doctor(Employee_number) ON DELETE SET NULL,
 13  CONSTRAINT fk_appt_nurse FOREIGN KEY(NEmployee_number) REFERENCES nurse(Enumber) ON DELETE SET NULL,
 14  CONSTRAINT fk_appt_vehicle FOREIGN KEY(vehicle_id) REFERENCES vehicle(VehicleID) ON DELETE SET NULL,
 15  CONSTRAINT fk_appt_treatment FOREIGN KEY(treatment_id) REFERENCES treatment_plan(Treatment_id) ON DELETE SET NULL);

Table created.
```

## 6.23 <*APPOINTMENT_BILL*> TABLE

CREATE TABLE appointment_bill(

bill_id NUMBER(10) PRIMARY KEY,

appointment_id NUMBER(10) UNIQUE,

total_cost NUMBER(10,2),

CONSTRAINT fk_apptbill_appt FOREIGN KEY(appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

```
SQL> CREATE TABLE appointment_bill(
  2  bill_id NUMBER(10) PRIMARY KEY,
  3  appointment_id NUMBER(10) UNIQUE,
  4  total_cost NUMBER(10,2),
  5  CONSTRAINT fk_apptbill_appt FOREIGN KEY(appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

Table created.
```

## 6.24 < *MEDICATION_APPOINTMENT*> TABLE

CREATE TABLE medication_appointment(

medication_id NUMBER(10)

, appointment_id NUMBER(10),

PRIMARY KEY(medication_id,appointment_id),

CONSTRAINT fk_medapp_med FOREIGN KEY(medication_id) REFERENCES medication(Medication_id) ON DELETE CASCADE,

CONSTRAINT fk_medapp_appt FOREIGN KEY(appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

```
SQL> CREATE TABLE medication_appointment(
  2  medication_id NUMBER(10),
  3  appointment_id NUMBER(10),
  4  PRIMARY KEY(medication_id,appointment_id),
  5  CONSTRAINT fk_medapp_med FOREIGN KEY(medication_id) REFERENCES medication(Medication_id) ON DELETE CASCADE,CONSTRAINT fk_medapp_appt FOREIGN KEY(appoin
tment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

Table created.
```

## 6.25 <*APPOINTMENT_SERVICE* > TABLE

CREATE TABLE appointment_service(

service_id NUMBER(10),

appointment_id NUMBER(10),

PRIMARY KEY(service_id,appointment_id),

CONSTRAINT fk_apptserv_service FOREIGN KEY(service_id) REFERENCES service(service_id) ON DELETE CASCADE,

CONSTRAINT fk_apptserv_appt FOREIGN KEY(appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

```
SQL> CREATE TABLE appointment_service(
  2  service_id NUMBER(10),
  3  appointment_id NUMBER(10),
  4  PRIMARY KEY(service_id,appointment_id),
  5  CONSTRAINT fk_apptserv_service FOREIGN KEY(service_id) REFERENCES service(service_id) ON DELETE CASCADE,CONSTRAINT fk_apptserv_appt FOREIGN KEY(appoint
ment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE);

Table created.
```

## 6.26 <*PATIENT_ALLERGY* > TABLE

CREATE TABLE PATIENT_ALLERGIES( Pfile_number NUMBER(7),

allergy VARCHAR2(255),

CONSTRAINT fk_patient_allergies FOREIGN KEY(Pfile_number) REFERENCES PATIENT(file_number) ON DELETE CASCADE, CONSTRAINT pk_patient_allergies PRIMARY KEY(Pfile_number,allergy));

```
SQL> CREATE TABLE PATIENT_ALLERGIES(
  2  Pfile_number NUMBER(7),
  3  allergy VARCHAR2(255),
  4  CONSTRAINT fk_patient_allergies FOREIGN KEY(Pfile_number) REFERENCES PATIENT(file_number) ON DELETE CASCADE,
  5  CONSTRAINT pk_patient_allergies PRIMARY KEY(Pfile_number,allergy));

Table created.
```

# 7 Constraints Script

| Business Rule | SQL Script | Table |
|---|---|---|
| Each patient can book multiple appointments | CREATE TABLE APPOINTMENT (<br>....<br>CONSTRAINT fk_appointment_patientnumber FOREIGN KEY (patient_number) REFERENCES patient(patient_number)<br>ON DELETE CASCADE,<br>...<br>); | APPOINTMENT |
| A doctor supervises multiple appointments | CREATE TABLE APPOINTMENT(<br>...<br>CONSTRAINT fk_appointment_employeenumber FOREIGN KEY (DEmployee_number) REFERENCES employee(employee_number)<br>ON DELETE SET NULL,<br>... | APPOINTMENT |

| | | |
|---|---|---|
| | );<br><br> | |
| Each doctor must work for one department | CREATE TABLE DOCTOR(<br>...<br><br>Department_name VARCHAR2(100) NOT NULL,<br><br>CONSTRAINT fk_doctor_department FOREIGN KEY(Department_name) REFERENCES DEPARTMENT(Department_name) ON DELETE CASCADE<br>....<br><br>); | DOCTOR |
| A doctor supervises multiple nurses | CREATE TABLE DOCTOR_NURSE (<br><br>Doctor_number NUMBER(6),<br><br>Nurse_number NUMBER(6),<br><br>CONSTRAINT PK_DOCTOR_NURSE<br>PRIMARY KEY (Doctor_number, Nurse_number),<br><br>CONSTRAINT FK_DOC_NURSE_DOCTOR<br>FOREIGN KEY (Doctor_number)<br>REFERENCES<br>EMPLOYEE(Employee_number),<br><br>CONSTRAINT FK_DOC_NURSE_NURSE<br>FOREIGN KEY (Nurse_number)<br>REFERENCES<br>EMPLOYEE(Employee_number)<br><br>); | DOCTOR_NURSE |
| A nurse is supervised by multiple doctors | CREATE TABLE DOCTOR_NURSE (<br><br>Doctor_number NUMBER(6),<br><br>Nurse_number NUMBER(6),<br><br>CONSTRAINT PK_DOCTOR_NURSE<br>PRIMARY KEY (Doctor_number, Nurse_number),<br><br>CONSTRAINT FK_DOC_NURSE_DOCTOR<br>FOREIGN KEY (Doctor_number)<br>REFERENCES<br>EMPLOYEE(Employee_number),<br><br>CONSTRAINT FK_DOC_NURSE_NURSE<br>FOREIGN KEY (Nurse_number)<br>REFERENCES<br>EMPLOYEE(Employee_number)<br><br>); | DOCTOR_NURSE |
| A nurse attends multiple appointments | CREATE TABLE APPOINTMENT (<br><br>....<br><br>CONSTRAINT fk_appointment_nemployeenumber FOREIGN KEY (NEmployee_number) REFERENCES employee(employee_number)<br>ON DELETE SET NULL<br>...<br><br>); | APPOINTMENT |
| Each nurse must work for one department. | CREATE TABLE NURSE(<br>... | NURSE |

| | | |
|---|---|---|
| | CONSTRAINT fk_nurse_department FOREIGN KEY (Dname) REFERENCES Department(Dname) ON DELETE CASCADE<br>....<br>); | |
| A department can have multiple doctors working for it. | CREATE TABLE DOCTOR(<br>...<br>Department_name VARCHAR2(100) NOT NULL,<br><br>CONSTRAINT fk_doctor_department FOREIGN KEY(Department_name) REFERENCES DEPARTMENT(Department_name) ON DELETE CASCADE,<br>....<br><br>); | DOCTOR |
| A department can have multiple nurses working for it. | CREATE TABLE NURSE(<br>...<br>CONSTRAINT fk_nurse_department FOREIGN KEY (Dname) REFERENCES Department(Dname) ON DELETE CASCADE<br>....<br>); | NURSE |
| Each service is included in many appointments. | CREATE TABLE APPOINTMENT_SERVICE (<br>Appointment_ID NUMBER(10),<br>Service_ID NUMBER(10),<br>CONSTRAINT pk_appointment_service PRIMARY KEY (Appointment_ID, Service_ID),<br>CONSTRAINT fk_appointment_service_appointmentid FOREIGN KEY (Appointment_ID) REFERENCES APPOINTMENT(Appointment_ID),<br>CONSTRAINT fk_appointment_service_serviceid FOREIGN KEY (Service_ID) REFERENCES SERVICE(Service_ID)<br>); | APPOINTMENT_SERVICE |
| Each appointment is booked for one patient. | CREATE TABLE APPOINTMENT (<br><br>....<br><br>CONSTRAINT fk_appointment_patientnumber FOREIGN KEY (patient_number) REFERENCES patient(patient_number)<br>ON DELETE CASCADE<br><br>...<br><br>); | APPOINTMENT |
| An appointment can have only one doctor. | CREATE TABLE APPOINTMENT(<br><br>...<br><br>CONSTRAINT fk_appointment_employeenumber FOREIGN KEY (DEmployee_number) REFERENCES employee(employee_number)<br>ON DELETE SET NULL,<br>...<br><br>); | APPOINTMENT |
| An appointment can have only one nurse. | CREATE TABLE APPOINTMENT (<br><br>....<br><br>CONSTRAINT fk_appointment_nemployeenumber FOREIGN KEY (NEmployee_number) REFERENCES employee(employee_number)<br>ON DELETE SET NULL,<br><br>... | APPOINTMENT |

| | | |
|---|---|---|
| | ); | |
| Each appointment must have one or more services. | CREATE TABLE APPOINTMENT_SERVICE ( Appointment_ID NUMBER(10), Service_ID NUMBER(10), CONSTRAINT pk_appointment_service PRIMARY KEY (Appointment_ID, Service_ID), CONSTRAINT fk_appointment_service_appointmentid FOREIGN KEY (Appointment_ID) REFERENCES APPOINTMENT(Appointment_ID), CONSTRAINT fk_appointment_service_serviceid FOREIGN KEY (Service_ID) REFERENCES SERVICE(Service_ID) ); | APPOINTMENT_SERVICE |
| Each appointment must have one bill. | CREATE TABLE appointment_bill ( appointment_id NUMBER(10), bill_id NUMBER(10), CONSTRAINT pk_appointment_bill PRIMARY KEY (appointment_id, bill_id), CONSTRAINT fk_appointment_bill_appointmentid FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE, CONSTRAINT fk_appointment_bill_billid FOREIGN KEY (bill_id) REFERENCES bill(bill_id) ON DELETE CASCADE<br><br>); | APPOINTMENT_BILL |
| Each appointment can use only one vehicle | CREATE TABLE APPOINTMENT(<br><br>...<br><br>CONSTRAINT fk_appointment_vehicleid FOREIGN KEY (veichel_id) REFERENCES veichel(veichel_id) ON DELETE SET NULL<br>...<br><br>); | APPOINTMENT |
| Each appointment has only one treatment plan. | CREATE TABLE APPOINTMENT(<br><br>...<br><br>CONSTRAINT fk_appointment_treatmentid FOREIGN KEY (treatmenet_id) REFERENCES treatment(treatmenet_id) ON DELETE SET NULL<br>...<br><br>); | APPOINTMENT |
| Each medication can be prescribed in one or more treatment plans, and for each prescription a dosage is recorded. | CREATE TABLE TREATMENT_PLAN_MEDICATION( Medication_ID NUMBER(10), Treatment_ID NUMBER(10), Dosage VARCHAR2(255), CONSTRAINT fk_treat_plan_medication_medicationid FOREIGN KEY (Medication_ID) REFERENCES MEDICATION(Medication_ID) ON DELETE CASSCADE, CONSTRAINT fk_treat_plan_medication_treatmentid | TREATMENT_PLAN_MEDICATION |

| | | |
|---|---|---|
| | FOREIGN KEY<br>(Treatment _ID)<br>REFERENCES<br>TREATMENT_PLAN (Treatment _ID)<br>ON DELETE CASSCADE,<br>CONSTRAINT<br>pk_treatment_plan_medication PRIMARY KEY(medication_id, treatment_id)<br>) | |
| Each vehicle can be assigned to multiple drivers. | CREATE TABLE VEHICLE_AND_DRIVER(<br><br>VehicleID NUMBER(10),<br>Dnumber NUMBER(10),<br>CONSTRAINT pk_vehicle_driver PRIMARY KEY (VehicleID, Dnumber),<br>CONSTRAINT fk_vehicle_driver FOREIGN KEY (VehicleID) REFERENCES Vehicle (VehicleID) ON DELETE CASCADE,<br>CONSTRAINT fk_driver_employee FOREIGN KEY (Dnumber) REFERENCES Vehicle_driver(Enumber) ON DELETE CASCADE<br><br>); | VEHICLE_AND_DRIVER |
| Each vehicle can be used for multiple appointments. | CREATE TABLE APPOINTMENT(<br><br>...<br><br>CONSTRAINT fk_appointment_vehicleid FOREIGN KEY (veichel_id) REFERENCES veichel(veichel_id)<br>ON DELETE SET NULL,<br>...<br><br>); | APPOINTMENT |
| Each driver can be assigned to multiple vehicles. | CREATE TABLE VEHICLE_AND_DRIVER(<br><br>VehicleID NUMBER(10),<br>Dnumber NUMBER(10),<br>CONSTRAINT pk_vehicle_driver PRIMARY KEY (VehicleID, Dnumber),<br>CONSTRAINT fk_vehicle_driver FOREIGN KEY (VehicleID) REFERENCES Vehicle (VehicleID) ON DELETE CASCADE,<br>CONSTRAINT fk_driver_employee FOREIGN KEY (Dnumber) REFERENCES Vehicle_driver(Enumber) ON DELETE CASCADE<br><br>); | VEHICLE_AND_DRIVER |
| Each bill must be associated with only one appointment. | CREATE TABLE appointment_bill ( appointment_id NUMBER(10), bill_id NUMBER(10), CONSTRAINT pk_appointment_bill PRIMARY KEY (appointment_id, bill_id), CONSTRAINT fk_appointment_bill_appointmentid FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE, CONSTRAINT fk_appointment_bill_billid FOREIGN KEY (bill_id) REFERENCES bill(bill_id) ON DELETE CASCADE<br><br>); | APPOINTMENT_BILL |

| | | |
|---|---|---|
| Each diagnosis is linked to one treatment plan. | CREATE TABLE TREATMENT_PLAN (<br>...<br>Dcode VARCHAR2(10),<br><br>fk_treatment_diagnosis FOREIGN KEY (Dcode) REFERENCES DIAGNOSIS(Code)<br>ON DELETE SET NULL<br>....<br>); | TREATMENT_PLAN |
| Each treatment plan can be on one or more appointments. | CREATE TABLE APPOINTMENT(<br><br>...<br><br>CONSTRAINT fk_appointment_treatmentid FOREIGN KEY (treatmenet_id) REFERENCES treatment(treatmenet_id)<br>ON DELETE SET NULL<br>...<br><br>); | APPOINTMENT |
| Each treatment plan can prescribe one or more medications, and for each prescription dosage is recorded. | CREATE TABLE TREATMENT_PLAN (<br>...<br>Dcode VARCHAR2(10),<br><br>fk_treatment_diagnosis FOREIGN KEY (Dcode) REFERENCES DIAGNOSIS(Code)<br>ON DELETE SET NULL<br>....<br>); | TREATMENT_PLAN |
| Each treatment plan must be related to one diagnosis. | CREATE TABLE TREATMENT_PLAN (<br>...<br>Dcode VARCHAR2(10),<br><br>fk_treatment_diagnosis FOREIGN KEY (Dcode) REFERENCES DIAGNOSIS(Code)<br>ON DELETE SET NULL<br>....<br>); | TREATMENT_PLAN |

# 8 Queries

1. List all home visits conducted within the specified period of time for patients who have more than one emergency contact.
2. List all patients who were prescribed more than 3 different medications across all their appointments.
3. List all the doctors and nurses who have supervised an appointment with a bill that cost more than 7000 SAR in the last 4 months.
4. List all patients whose treatment duration is longer than the average treatment duration of patients with the same diagnosis.
5. List all patients who were treated by more than one doctor across different visits and received at least one prescription each time.

## 8.1 *<Title of query1>*

**Query in natural language (ENGLISH)**

List all home visits conducted within the specified period of time for patients who have more than one emergency contact.

**SQL script**

```
SELECT a.appointment_id, a.appointment_date, a.appointment_time, a.patient_number
FROM appointment a
JOIN treatment_plan t ON a.treatment_id = t.treatment_id
JOIN appointment_service aps ON a.appointment_id = aps.appointment_id
JOIN service s ON aps.service_id = s.service_id
WHERE LOWER(s.name) LIKE '%home%'
  AND a.appointment_date BETWEEN TO_DATE('2024-01-01', 'YYYY-MM-DD')
                             AND TO_DATE('2024-12-31', 'YYYY-MM-DD')
  AND a.patient_number IN (
    SELECT patient_number
    FROM patient_emergency
    GROUP BY patient_number
    HAVING COUNT(*) > 1
);
```

**Caption of the first five rows of the output**

```
SQL> SELECT a.appointment_id, a.appointment_date, a.appointment_time, a.patient_number
  2  FROM appointment a
  3  JOIN treatment_plan t ON a.treatment_id = t.treatment_id
  4  JOIN appointment_service aps ON a.appointment_id = aps.appointment_id
  5  JOIN service s ON aps.service_id = s.service_id
  6  WHERE LOWER(s.name) LIKE '%home%'
  7  AND a.appointment_date BETWEEN TO_DATE('2024-01-01', 'YYYY-MM-DD') AND TO_DATE('2024-12-31', 'YYYY-MM-DD')
  8  AND a.patient_number IN (
  9      SELECT patient_number
 10      FROM patient_emergency
 11      GROUP BY patient_number
 12      HAVING COUNT(*) > 1
 13  );

no rows selected
```

## 8.2 *<Title of query1>*

**Query in natural language (ENGLISH)**

List all patients who were prescribed more than 3 different medications across all their appointments.

**SQL script**

```
SELECT p.file_number, p.first_name, p.last_name
FROM patient p
WHERE (
  SELECT COUNT(DISTINCT ma.medication_id)
  FROM appointment a
  JOIN medication_appointment ma ON a.appointment_id = ma.appointment_id
  WHERE a.patient_number = p.file_number
) > 3;
```

**Caption of the first five rows of the output**

```
SQL> SELECT p.file_number, p.first_name, p.last_name
  2  FROM patient p
  3  WHERE (
  4      SELECT COUNT(DISTINCT ma.medication_id)
  5      FROM appointment a
  6      JOIN medication_appointment ma ON a.appointment_id = ma.appointment_id
  7      WHERE a.patient_number = p.file_number
  8  ) > 3;

FILE_NUMBER FIRST_NAME
----------- --------------------------------------------------
LAST_NAME
--------------------------------------------------
       1002 Bob
Johnson
```

## 8.3 *<Title of query1>*

**Query in natural language (ENGLISH)**

List all the doctors and nurses who have supervised an appointment with a bill that cost more than 7000 SAR in the last 4 months.

**SQL script**

```
SELECT DISTINCT
  e.first_name || ' ' || e.last_name AS full_name,
  b.total_cost AS bill_amount
FROM appointment a
JOIN appointment_bill ab ON a.appointment_id = ab.appointment_id
JOIN bill b ON ab.bill_id = b.bill_id
```

JOIN employee e ON e.employee_number IN (a.DEmployee_number, a.NEmployee_number)

WHERE b.total_cost > 7000

AND a.appointment_date >= ADD_MONTHS(SYSDATE, -4);

**Caption of the first five rows of the output**

```
SQL> SELECT DISTINCT
  2      e.first_name || ' ' || e.last_name AS full_name,
  3      b.total_cost AS bill_amount
  4  FROM appointment a
  5  JOIN appointment_bill ab ON a.appointment_id = ab.appointment_id
  6  JOIN bill b ON ab.bill_id = b.bill_id
  7  JOIN employee e ON e.employee_number IN (a.DEmployee_number, a.NEmployee_number)
  8  WHERE b.total_cost > 7000
  9  AND a.appointment_date >= ADD_MONTHS(SYSDATE, -4);

FULL_NAME
--------------------------------------------------------------------------------
BILL_AMOUNT
-----------
Sara Alghamdi
       7070

Aisha Alotaibi
       7070

Sara Alzahrani
      20000


FULL_NAME
--------------------------------------------------------------------------------
BILL_AMOUNT
-----------
Aisha Alzahrani
      20000

Faisal Alharbi
      13000

Khalid Alotaibi
      13000


6 rows selected.
```

## 8.4 *<Title of query1>*

**Query in natural language (ENGLISH)**

List all patients whose treatment duration is longer than the average treatment duration of patients with the same diagnosis.

**SQL script**

```
SELECT DISTINCT p.file_number, p.first_name, p.last_name

FROM patient p

JOIN appointment a ON p.file_number = a.patient_number

JOIN treatment_plan t ON a.treatment_id = t.treatment_id

WHERE t.duration > (

  SELECT AVG(t2.duration)

  FROM treatment_plan t2

  WHERE t2.dcode = t.dcode

);
```

**Caption of the first five rows of the output**

```
SQL> SELECT DISTINCT p.file_number, p.first_name, p.last_name
  2  FROM patient p
  3  JOIN appointment a ON p.file_number = a.patient_number
  4  JOIN treatment_plan t ON a.treatment_id = t.treatment_id
  5  WHERE t.duration > (
  6      SELECT AVG(t2.duration)
  7      FROM treatment_plan t2
  8      WHERE t2.dcode = t.dcode
  9  );

FILE_NUMBER FIRST_NAME
----------- --------------------------------------------------
LAST_NAME
--------------------------------------------------
       9002 Sara
Salem
```

## 8.5 *<Title of query1>*

**Query in natural language (ENGLISH)**

List all patients who were treated by more than one doctor across different visits and received at least one prescription each time.

**SQL script**

```
SELECT patient_number
FROM (
  SELECT a.patient_number, COUNT(DISTINCT a.DEmployee_number) AS
unique_doctors
  FROM appointment a
  JOIN medication_appointment ma ON a.appointment_id = ma.appointment_id
  GROUP BY a.patient_number
)
WHERE unique_doctors > 1;
```

**Caption of the first five rows of the output**

```
SQL> SELECT patient_number
  2  FROM (
  3      SELECT a.patient_number, COUNT(DISTINCT a.DEmployee_number) AS unique_doctors
  4      FROM appointment a
  5      JOIN medication_appointment ma ON a.appointment_id = ma.appointment_id
  6      GROUP BY a.patient_number
  7  )
  8  WHERE unique_doctors > 1;

PATIENT_NUMBER
--------------
          9001
```

# APPENDIX

```
SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Pediatrics');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Cardiology');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Oncology');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Orthopedics');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Surgery');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Emergency');

1 row created.

SQL> INSERT INTO DEPARTMENT (Dname) VALUES ('Gastroenterology');

1 row created.
```

```
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (1, 'Aspirin', 'Take one tablet daily', 'Y');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (2, 'Ibuprofen', 'Take as needed for pain', 'N');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (3, 'Lisinopril', 'Take one tablet daily', 'Y');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (4, 'Metformin', 'Take two tablets with meals', 'Y');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (5, 'Atorvastatin', 'Take one tablet at night', 'N');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (6, 'Levothyroxine', 'Take on an empty stomach', 'Y');

1 row created.

SQL>
SQL>
SQL> INSERT INTO MEDICATION (Medication_id, Name, Usage_instruction, Refill_availability) VALUES (7, 'Simvastatin', 'Take one tablet in the evening', 'Y');


1 row created.
```

```
SQL> INSERT INTO appointment_bill (appointment_id, bill_id) VALUES (5, 5);

1 row created.

SQL> INSERT INTO appointment_bill (appointment_id, bill_id) VALUES (6, 6);

1 row created.

SQL> INSERT INTO appointment_bill (appointment_id, bill_id) VALUES (7, 7);

1 row created.

SQL> INSERT INTO appointment_bill (appointment_id, bill_id) VALUES (8, 8);

1 row created.

SQL> INSERT INTO appointment_bill (appointment_id, bill_id) VALUES (9, 9);

1 row created.
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (1, 300002);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (2, 300003);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (2, 300004);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (3, 300005);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (3, 300006);

1 row created.

SQL>
SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (4, 300001);

1 row created.

SQL>
SQL>
SQL>
SQL> INSERT INTO VEHICLE_AND_DRIVER (VehicleID, Dnumber) VALUES (4, 300007);

1 row created.
SQL> INSERT INTO PATIENT_EMERGENCY (Patient_number, Relationship, Name, Phone_number) VALUES (1002, 'Parent', 'Alice Johnson', '555-8765');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_EMERGENCY (Patient_number, Relationship, Name, Phone_number) VALUES (1003, 'Child', 'Bob Brown', '555-4321');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_EMERGENCY (Patient_number, Relationship, Name, Phone_number) VALUES (1004, 'Friend', 'Mike Prince', '555-6789');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_EMERGENCY (Patient_number, Relationship, Name, Phone_number) VALUES (1005, 'Partner', 'Emily Hunt', '555-0987');

1 row created.
```

```
SQL> INSERT INTO medication_appointment (medication_id, appointment_id) VALUES (5, 2);

1 row created.

SQL> INSERT INTO medication_appointment (medication_id, appointment_id) VALUES (6, 2);

1 row created.

SQL> INSERT INTO medication_appointment (medication_id, appointment_id) VALUES (7, 2);

1 row created.

SQL> INSERT INTO medication_appointment (medication_id, appointment_id) VALUES (8, 2);

1 row created.
```

```
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (1, 'Operational', 1234567890);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (2, 'Operational', 2345678901);

1 row created.

SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (3, 'Under Maintenance', 3456789012);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (4, 'Operational', 4567890123);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (5, 'Out of Service', 5678901234);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (6, 'Operational', 6789012345);

1 row created.

SQL>
SQL>
SQL> INSERT INTO VEHICLE (VehicleID, OperationalStatus, PlateNumber) VALUES (7, 'Operational', 7890123456);

1 row created.
```

```
SQL> INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300001);

1 row created.

SQL>
SQL> INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300002);

1 row created.

SQL>
SQL>  INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300003);

1 row created.

SQL>
SQL> INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300004);

1 row created.

SQL>
SQL> INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300005);

1 row created.

SQL>
SQL> INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300006);

1 row created.

SQL>
SQL>  INSERT INTO VEHICLE_DRIVER (Enumber) VALUES (300007);

1 row created.
```

```
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (8, 5);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (9, 5);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (10, 6);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (1, 7);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (2, 8);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (3, 9);

1 row created.

SQL>
SQL> INSERT INTO appointment_service (service_id, appointment_id) VALUES (4, 10);

1 row created.
```

```
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (2, 'Cash', 'Pending', 20000.00);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (3, 'Insurance', 'Paid', 13000.00);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (4, 'Debit Card', 'Failed', 120.50);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (5, 'PayPal', 'Paid', 75.25);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (6, 'Bank Transfer', 'Pending', 450.00);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (7, 'Credit Card', 'Paid', 250.75);

1 row created.

SQL>
SQL>
SQL> INSERT INTO BILL (bill_id, payment_method, payment_status, total_cost) VALUES (8, 'Cash', 'Cancelled', 7070.00);

1 row created.
```

```
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1001, 'Diabetes');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1002, 'Asthma');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1003, 'Cardiovascular Disease');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1004, 'Hypothyroidism');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1005, 'Anxiety');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1006, 'None');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_CHRONIC_ILLNESS (Pfile_number, Chronic_illness) VALUES (1007, 'Arthritis');

1 row created.
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100003, 'MD in Pediatrics from Stanford University');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100004, 'MD in Orthopedics from Mayo Clinic Alix School of Medicine');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100005, 'MD in Dermatology from University of California, San Francisco');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100006, 'MD in Psychiatry from University of Pennsylvania');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100007, 'MD in Gastroenterology from University of Michigan');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_ACADEMIC_DEGREE (Doctor_ID, Academic_degree) VALUES (100008, 'MD in Oncology from Duke University');

1 row created.

SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D002', 'Diabetes Mellitus', 'Chronic condition affecting glucose metabolism.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D003', 'Asthma', 'Chronic respiratory condition with wheezing.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D004', 'Cholesterol Disorder', 'Abnormal lipid levels in the blood.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D005', 'Heart Disease', 'A range of conditions affecting the heart.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D006', 'Anemia', 'Deficiency of red blood cells or hemoglobin.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D007', 'Osteoarthritis', 'Degenerative joint disease causing pain and stiffness.');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DIAGNOSIS (Code, Name, Clinical_note) VALUES ('D008', 'Allergy', 'Immune response to foreign substances.');

1 row created.
```

```
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100001, 'Cardiology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100002, 'Neurology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100003, 'Pediatrics');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100004, 'Orthopedics');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100005, 'Dermatology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100006, 'Psychiatry');

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR (Employee_number, Specialization) VALUES (100007, 'Gastroenterology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO EMPLOYEE_AVAILABLE_SCHEDULE (Employee_number, Day, Year, Time, Duration) VALUES (200006, 'Saturday', 2023, TO_TIMESTAMP('12:00:00', 'HH24:M
I:SS'), 5);

1 row created.

SQL>
SQL>
SQL> INSERT INTO EMPLOYEE_AVAILABLE_SCHEDULE (Employee_number, Day, Year, Time, Duration) VALUES (300004, 'Wednesday', 2023, TO_TIMESTAMP('06:00:00', 'HH24:
MI:SS'), 4);

1 row created.

SQL>
SQL>
SQL> INSERT INTO EMPLOYEE_AVAILABLE_SCHEDULE (Employee_number, Day, Year, Time, Duration) VALUES (300004, 'Thursday', 2023, TO_TIMESTAMP('11:00:00', 'HH24:M
I:SS'), 3);

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE_AVAILABLE_SCHEDULE (Employee_number, Day, Year, Time, Duration) VALUES (300005, 'Friday', 2023, TO_TIMESTAMP('07:30:00', 'HH24:MI:
SS'), 5);

1 row created.

SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100001, 'E100001', 'Ali', 'Alqahtani', TO_DATE('1980-05-10', 'YYYY-MM-DD'), 'M', '0551000001', 'ali.alqahtani@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100002, 'E100002', 'Sara', 'Alzahrani', TO_DATE('1985-08-22', 'YYYY-MM-DD'), 'F', '0551000002', 'sara.alzahrani@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100003, 'E100003', 'Faisal', 'Alharbi', TO_DATE('1979-11-30', 'YYYY-MM-DD'), 'M', '0551000003', 'faisal.alharbi@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100004, 'E100004', 'Laila', 'Alshehri', TO_DATE('1990-02-18', 'YYYY-MM-DD'), 'F', '0551000004', 'laila.alshehri@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100005, 'E100005', 'Omar', 'Aldossari', TO_DATE('1988-07-07', 'YYYY-MM-DD'), 'M', '0551000005', 'omar.aldossari@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (100006, 'E100006', 'Noura', 'Alghamdi', TO_DATE('1982-04-03', 'YYYY-MM-DD'), 'F', '0551000006', 'noura.alghamdi@example.com');

1 row created.
```

```
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200001, 'E200001', 'Noura', 'Alamri', TO_DATE('1987-01-10', 'YYYY-MM-DD'), 'F', '0552000001', 'noura.alamri@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200002, 'E200002', 'Aisha', 'Alzahrani', TO_DATE('1992-02-14', 'YYYY-MM-DD'), 'F', '0552000002', 'aisha.alzahrani@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200003, 'E200003', 'Khalid', 'Alotaibi', TO_DATE('1989-03-20', 'YYYY-MM-DD'), 'M', '0552000003', 'khalid.alotaibi@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200004, 'E200004', 'Fatima', 'Alqahtani', TO_DATE('1985-04-18', 'YYYY-MM-DD'), 'F', '0552000004', 'fatima.alqahtani@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200005, 'E200005', 'Laila', 'Alshehri', TO_DATE('1990-05-22', 'YYYY-MM-DD'), 'F', '0552000005', 'laila.alshehri@example.com');

1 row created.

SQL>
SQL> INSERT INTO EMPLOYEE (Employee_number, ID, First_name, Last_name, Date_of_birth, Gender, Phone_number, Email)
  2  VALUES (200006, 'E200006', 'Ali', 'Almutairi', TO_DATE('1984-06-16', 'YYYY-MM-DD'), 'M', '0552000006', 'ali.almutairi@example.com');

1 row created.
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100003, 200001);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100004, 200004);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100005, 200008);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100006, 200006);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100007, 200007);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100008, 200003);

1 row created.

SQL>
SQL>
SQL> INSERT INTO DOCTOR_NURSE (Doctor_number, Nurse_number) VALUES (100009, 200010);

1 row created.
```

```
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200003, 'Oncology Certified Nurse');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200004, 'Orthopedic Certified Nurse');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200005, 'Surgical Nurse Practitioner');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200006, 'Emergency Room Nurse');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200007, 'Gastroenterology Nurse Specialist');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200008, 'Dermatology Nurse Practitioner');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE_QUALIFICATION (Nurse_number, Qualification) VALUES (200009, 'Neurology Nurse Specialist');

1 row created.
```

```
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200003, 'Oncology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200004, 'Orthopedics');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200005, 'Surgery');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200006, 'Emergency');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200007, 'Gastroenterology');

1 row created.

SQL>
SQL>
SQL> INSERT INTO NURSE (Enumber, Dname) VALUES (200008, 'Dermatology');

1 row created.
```

```
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1001, 'Peanuts');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1001, 'Shellfish');

1 row created.

SQL>
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1002, 'Penicillin');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1003, 'Latex');

1 row created.

SQL>
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1004, 'Dust mites');

1 row created.

SQL>
SQL>
SQL> INSERT INTO PATIENT_ALLERGIES (Pfile_number, Allergy) VALUES (1005, 'Pollen');

1 row created.
SQL> INSERT INTO PATIENT (File_number, Patient_ID, First_name, Last_name, Date_of_birth, Email, Phone_number, Gender, District, Streat, Building_number, Pos
tal_code, Eligibility_criteria)
  2  VALUES (1001, 20001, 'Alice', 'Smith', TO_DATE('1990-05-15', 'YYYY-MM-DD'), 'alice.smith@example.com', '1234567890', 'F', 'Downtown', 'Main St', '101',
 '12345', 'Eligible');

1 row created.

SQL>
SQL> INSERT INTO PATIENT (File_number, Patient_ID, First_name, Last_name, Date_of_birth, Email, Phone_number, Gender, District, Streat, Building_number, Pos
tal_code, Eligibility_criteria)
  2  VALUES (1002, 20002, 'Bob', 'Johnson', TO_DATE('1985-03-22', 'YYYY-MM-DD'), 'bob.johnson@example.com', '2345678901', 'M', 'Uptown', 'Second St', '202',
 '23456', 'Eligible');

1 row created.

SQL>
SQL> INSERT INTO PATIENT (File_number, Patient_ID, First_name, Last_name, Date_of_birth, Email, Phone_number, Gender, District, Streat, Building_number, Pos
tal_code, Eligibility_criteria)
  2  VALUES (1003, 20003, 'Charlie', 'Brown', TO_DATE('1978-11-30', 'YYYY-MM-DD'), 'charlie.brown@example.com', '3456789012', 'M', 'Midtown', 'Third St', '3
03', '34567', 'Not Eligible');

1 row created.

SQL>
SQL> INSERT INTO PATIENT (File_number, Patient_ID, First_name, Last_name, Date_of_birth, Email, Phone_number, Gender, District, Streat, Building_number, Pos
tal_code, Eligibility_criteria)
  2  VALUES (1004, 20004, 'Diana', 'Prince', TO_DATE('1992-07-12', 'YYYY-MM-DD'), 'diana.prince@example.com', '4567890123', 'F', 'Westside', 'Fourth St', '4
04', '45678', 'Eligible');

1 row created.

SQL>
SQL> INSERT INTO PATIENT (File_number, Patient_ID, First_name, Last_name, Date_of_birth, Email, Phone_number, Gender, District, Streat, Building_number, Pos
tal_code, Eligibility_criteria)
  2  VALUES (1005, 20005, 'Ethan', 'Hunt', TO_DATE('1980-01-25', 'YYYY-MM-DD'), 'ethan.hunt@example.com', '5678901234', 'M', 'Eastside', 'Fifth St', '505',
'56789', 'Eligible');

1 row created.
```